Highlights

Diagnose locally, escalate rarely: bandwidth-efficient plant disease recognition with foundation models on embedded edge–cloud hardware

Youssef Natij, Hajar El Karch, Abdelkader Mezouari

- An edge model escalates only uncertain field images to a foundation-model ensemble
- 18.6 % escalation retains 94.6 % of cloud accuracy with 5.4× less uplink
- Edge-model capacity, not router complexity, sets the trade-off
- A 0.9-billion-parameter ensemble runs on an end-of-life 4 GB Jetson Nano
- Four-farm emulation on the devices: 5.4× less uplink, 8× with a distilled edge

Diagnose locally, escalate rarely: bandwidth-efficient plant disease recognition with foundation models on embedded edge–cloud hardware

Youssef Natij[a,*], Hajar El Karch[b], Abdelkader Mezouari[a]

[a] *Scientific Research and Innovation Laboratory, Higher School of Technology, Ibn Tofail University, Campus Universitaire, B.P. 242, 14000, Kenitra, Morocco*
[b] *Laboratory of Advanced Systems Engineering, National School of Applied Engineering, Ibn Tofail University, Campus Universitaire, B.P. 241, 14000, Kenitra, Morocco*

Abstract

Image-based plant disease diagnosis in the field faces a dilemma: lightweight models run on low-cost farm devices but misdiagnose about one field photograph in three, whereas foundation-model ensembles approach expert accuracy but need every photograph uploaded over weak, metered rural links. We study selective edge–cloud inference as the middle ground, evaluated by uplink traffic, the binding constraint on farms. On PlantWild (18 542 in-the-wild images, 89 classes), a MobileNetV3-Large edge classifier escalates an image to a three-backbone foundation-model ensemble only when its two most probable classes are closer than a validation-calibrated threshold. The rule escalates 18.6 % of test images and retains 94.6 % of always-cloud accuracy, cutting uplink traffic 5.4-fold. Edge-model capacity, not router sophistication, sets this operating point: a weaker edge model needs 43.5 % escalation for the same target, and a cloud-supervised verifier ranks useful escalations worse than the margin (AUC 0.731 vs. 0.810) and adds an always-on message. We deploy the system on a Raspberry Pi 5 and an end-of-life 4 GB Jetson Nano, porting the ensemble to framework-free, chunk-by-chunk TensorRT inference. In an emulated seven-day, four-farm deployment, it uploads 5.4× less than always-cloud, answers on the farm in 77 ms (median) and confirms

*Corresponding author.
Email addresses: youssef.natij@uit.ac.ma (Youssef Natij), hajar.elkarch@uit.ac.ma (Hajar El Karch), abdelkader.mezouari1@uit.ac.ma (Abdelkader Mezouari)

each escalated image with 6.9 s of Nano compute. Distilling the ensemble into the same edge model, at no cost on the device, raises the measured saving to 8.0× and cuts device energy per diagnosis by a third. Selective escalation thus brings foundation-model accuracy to rural links on commodity hardware.

Keywords: plant disease recognition, edge computing, selective inference, foundation models, bandwidth efficiency, embedded systems

1. Introduction

Pathogens and pests destroy a substantial fraction of the global harvest of major food crops every year [1], and early, accurate diagnosis is what allows a grower to respond before an outbreak spreads. Smartphone and camera-based diagnosis has therefore become one of the most studied applications of deep learning in agriculture [2–4]. Yet the field has long been flattered by laboratory benchmarks: models that exceed 99 % accuracy on curated leaf images such as PlantVillage [5] lose much of that accuracy on photographs taken in real fields, where backgrounds are cluttered, lighting is uncontrolled and symptoms are small [6–9]. The PlantWild benchmark [10] made this gap measurable at scale, with 18 542 in-the-wild images spanning 89 crop–disease classes.

Two families of models now address this setting, and they sit at opposite ends of a deployment trade-off. Frozen *foundation-model* features — self-supervised DINOv2 and DINOv3 and language-supervised CLIP [11–13] — are remarkably robust to field conditions; in prior work we showed that fusing three such backbones reaches 80.17 % top-1 on PlantWild without any text supervision [14]. Such ensembles, however, total close to a billion parameters and must run on a server. Lightweight convolutional networks such as MobileNetV3 [15] run comfortably on a farm-grade device but fall more than ten points short in the wild. A grower must thus choose between an accurate model that requires connectivity and a local model that is frequently wrong.

Connectivity is precisely what rural agriculture lacks. Cellular uplinks on farms are slow, intermittent and metered [16, 17], and every photograph sent to a server costs airtime, energy and money. Uploading every image is the most expensive way to use a scarce resource, yet it is the implicit assumption of most cloud-based diagnosis systems. Edge computing [18]

suggests the alternative: process data where it is produced and send only what cannot be handled locally.

This paper studies *selective escalation*: an edge model diagnoses every image locally and forwards an image to the cloud ensemble only when it is uncertain. The idea is old in machine learning — classifier cascades [19], the reject option [20] and early exiting [21] all exploit it — but its evaluation has centred on computation and latency. For connected agriculture the binding constraint is the uplink, so we make *bytes uploaded per diagnosis* the primary cost axis and ask three questions:

1. How much uplink traffic can selective escalation save while retaining most of the cloud ensemble's accuracy on in-the-wild images?
2. What determines the achievable operating point: the edge model, or the sophistication of the escalation rule?
3. Does the approach survive contact with real embedded hardware, including a cloud node small enough to sit on a farm or in a cooperative?

Contributions.

- **A bandwidth-centred evaluation of selective edge–cloud inference for plant disease recognition.** A MobileNetV3-Large edge model with a one-line margin rule, its threshold calibrated on validation data, retains 94.6 % of the ensemble's accuracy on held-out test images while escalating 18.6 % of them, a 5.4-fold reduction in uplink traffic (Section 6.2).

- **Evidence that edge capacity, not router cleverness, sets the operating point.** Upgrading the edge model halves the required escalation rate (43.5 % → 20.3 %), and distilling the ensemble into the same edge model — once, before deployment, with no change on the device — raises the uplink saving measured on the devices from 5.4- to 8.0-fold, and from 5.4- to 7.0-fold at equal accuracy, whereas a learned edge-side router saves only two points of escalation and a cloud-supervised verifier does worse than the scalar margin while adding a permanent message overhead (Sections 6.1–6.4 and 6.7).

- **A recipe for running foundation models on end-of-life embedded hardware.** The edge runs on a Raspberry Pi 5, and the 0.9-billion-parameter ensemble runs on a 4 GB Jetson Nano whose last software

release predates every framework the ensemble needs. Five principles make this possible — decoupling the model from its framework, compiling within the memory ceiling, choosing precision by measurement, chunk-major batched inference and stage-by-stage verification — and the board serves an escalated image with 49 J of energy, with board memory peaking at 2.32 GB (Section 5 and Appendix A).

- **A trace-driven, hardware-in-the-loop emulation of a multi-farm week.** Four virtual farms with distinct crop portfolios play back a seven-day capture schedule on the real devices, yielding measured per-image latency, per-farm upload volume, cloud load and device energy, for the original and the distilled edge model (Sections 6.6 and 6.7).

2. Related work

2.1. Plant disease recognition in the wild

Early deep-learning studies reported near-perfect accuracy on laboratory images [3, 4], but subsequent work traced much of that performance to dataset bias and showed sharp drops under field conditions [6–8]. Field-oriented datasets such as PlantDoc [22] and, at a larger scale, PlantWild [10] now drive the area, together with multimodal baselines that inject textual disease descriptions [23]. Recent work combines foundation backbones [24, 25] or compresses ensembles for deployment [26]. Our cloud model is the three-backbone QualityGate fusion of **(author?)** [14]; this paper does not propose a new recogniser but asks how to deliver such a recogniser's accuracy to farms that cannot afford to upload every image.

2.2. Edge–cloud collaborative inference

Collaborative inference distributes a model's computation between a device and a server. Neurosurgeon partitions a single network at the layer that minimises latency or energy [27]; distributed DNNs and early-exit networks attach classifiers to intermediate layers so that easy inputs leave early [21, 28]; split computing compresses the intermediate tensor sent over the network [29]. These methods co-design a single model. We study the complementary *model-cascade* setting, in which the edge and cloud models are independent and the cloud model is a frozen ensemble that cannot be partitioned or retrained for the edge — the practical situation when the best available recogniser is a set of foundation backbones.

2.3. Cascades, selective prediction and escalation rules

Deciding which inputs deserve the expensive model is a selective prediction problem [20, 30]. The maximum softmax probability is a strong baseline for detecting likely errors [31], although modern networks are miscalibrated [32]. Learned deferral policies have been proposed for cascades [33] and for edge devices that query a cloud model [34]. We compare scalar uncertainty signals against both edge-side learned routers and a cloud-supervised verifier, and we account for the bandwidth that a learned verifier itself consumes — a cost that latency-centred evaluations omit.

2.4. Edge AI in agriculture

Surveys of deep learning in agriculture [2] and of smart farming [16] identify connectivity and on-farm computation as recurring obstacles, and edge computing has been argued to be a natural fit for agricultural data [17]. The disease-recognition studies discussed in Section 2.1, however, evaluate either a server-side or a fully on-device classifier and report accuracy and, at most, computational cost; none of them measures the uplink traffic that a deployment would generate. Our deployment also differs in placing the large model on an embedded node rather than in a data centre.

3. Materials and methods

3.1. Dataset and protocol

We use PlantWild [10]: 18 542 field photographs of 89 classes spanning healthy and diseased leaves of more than thirty plant species. We follow the official split procedure of **(author?)** [23] (fixed seed, 80 % train-validation and 20 % test, with 12.5 % of train-validation held out for validation), giving 13 045 training, 1 820 validation and 3 677 test images. All accuracies are top-1 on the 3 677-image test split. Training, offline evaluation and the ONNX export ran on a workstation with an NVIDIA GeForce RTX 3070 (8 GB), an AMD Ryzen 7 7700 CPU and 64 GB of RAM.

3.2. Cloud model: frozen foundation-backbone fusion

The cloud model is the QualityGate fusion of **(author?)** [14] (Fig. 1, right). Three frozen backbones each produce a global image descriptor: DINOv2 ViT-L/14 at 336 px [11], DINOv3 ViT-L/16 at 336 px [12], and the CLIP ViT-L/14 image encoder at 224 px [13]. Each descriptor is ℓ_2-normalised, the three are concatenated into a 2 816-dimensional vector and

the result is normalised again, giving $\hat{x}$. The QualityGate head mixes a linear classifier with a prototype classifier (K=8 prototypes $p_{c,k}$ per class) through a learned per-class gate:

$$z_c = g_c \left(w_c^\top \hat{x} + b_c \right) + (1 - g_c) \, \frac{1}{T} \max_k \cos(\hat{x}, p_{c,k}) \,, \qquad g_c = \sigma(\gamma_c), \tag{1}$$

with a learned temperature T. Only the head is trained (2.3 M parameters, 50 epochs); the backbones total about 0.9 B frozen parameters. The model reaches 80.17 % top-1 and 77.49 % macro-F1 on the test split, and we refer to it as *always-cloud*.

3.3. Edge models

We consider three ImageNet-pretrained convolutional networks that fit a farm-grade CPU: MobileNetV3-Small, MobileNetV3-Large [15] and EfficientNet-B0 [35]. Each is fine-tuned end to end at 224×224 for 30 epochs with AdamW [36] (learning rate $5{\times}10^{-4}$, weight decay 10^{-4}, batch 64, cosine schedule with 5 % linear warm-up), focal loss [37] with γ=2 and label smoothing 0.1, and random resized crops (scale 0.6–1.0), horizontal flips and colour jitter. The models are trained on 11 778 of the 13 045 training images; the remaining 1 267 (10 % of each class, rounded down) select the checkpoint with the best macro-F1, so that the 1 820 validation images remain unseen until the escalation threshold is calibrated on them.

3.4. Margin-based escalation

Let $p_{(1)} \geq p_{(2)}$ be the two largest softmax probabilities of the edge model for image x. The edge escalates x to the cloud if and only if its *margin* is small,

$$m(x) = p_{(1)}(x) - p_{(2)}(x) < \tau, \tag{2}$$

and otherwise returns its own prediction. The rule has a single parameter, costs one subtraction, and requires no training. Sweeping τ traces an accuracy–escalation curve; we also evaluate the maximum probability $p_{(1)}$ and the predictive entropy as alternative signals, two learned edge-side routers, and an *oracle* that escalates exactly those images the edge gets wrong and the cloud gets right — an upper bound for any escalation rule. The learned routers are perceptrons with two hidden layers of 128 units (layer normalisation of the input, GELU, dropout 0.4, AdamW with learning rate 10^{-3} and weight decay 10^{-3}, cosine schedule) that predict a *useful escalation*

(edge wrong, cloud right) from the edge model's softmax and its three scalar scores, and, for the second router, also from the 1 024-dimensional DINOv2 descriptor of the image. They are trained on the validation split, with the number of epochs chosen by five-fold cross-validation within it, and evaluated on the test split.

Threshold calibration. The deployed threshold is fixed before any test image is seen: τ is the smallest value whose *validation* accuracy reaches 95 % of the cloud model's validation accuracy. For MobileNetV3-Large this gives $\tau = 0.138$, which escalates 19.3 % of validation images. The same τ is then applied unchanged to the test split and to the deployment. Because τ is estimated from 1 820 images, we also report its sampling variability by recalibrating it on 2 000 bootstrap resamples of the validation split (Section 6.2). Full accuracy–escalation curves, which do not depend on any particular τ, are used only to compare routers and edge models.

3.5. Bandwidth model

Let r be the escalation rate and S the mean size of an escalated image. The mean uplink traffic per diagnosis is

$$\bar{B}_{\text{margin}} = r\,S, \qquad \bar{B}_{\text{verifier}} = M + r\,S, \qquad \bar{B}_{\text{cloud}} = S, \tag{3}$$

where M is the size of any message a router sends for *every* image (Section 3.6). The reduction factor of margin escalation over always-cloud is therefore $\bar{B}_{\text{cloud}}/\bar{B}_{\text{margin}} = 1/r$, independent of S. To express traffic in absolute terms we set S to the measured mean size of the escalation payload used in the deployment (Section 4.3): a 448 px JPEG at quality 90, $S = 60.2$ KB over the 3 677 test images. Offline predictions and on-device measurements (Section 6.6) are therefore in the same units. Sizes use binary prefixes throughout (1 KB = 1024 bytes, 1 MB = 1024 KB).

3.6. Cloud-supervised verifier

To test whether cloud-side information can improve escalation, we train a verifier: a multilayer perceptron that runs on the cloud and receives, for every image, the edge model's 1 280-dimensional penultimate embedding, its 89-dimensional softmax and its uncertainty scores. It is trained to predict one of three targets — *useful escalation* (edge wrong, cloud right), *cloud disagrees* with the edge, or *cloud correct* — and the image is escalated when the verifier's score exceeds a threshold. The perceptron has two hidden layers

(512 and 256 units for the first target, 256 and 128 for the others; layer normalisation of the input, GELU, dropout 0.3) and is trained with a class-weighted binary cross-entropy (AdamW, learning rate 3×10^{-4}, weight decay 10^{-4}, 40 epochs) on the validation split, of which 20 % select the checkpoint by AUC; it is evaluated on the test split. Because the verifier needs the embedding of every image, it adds an always-on message of $M = 2 \times (1\,280 + 89)$ bytes $= 2.67$ KB in half precision; the three scalar scores are functions of the softmax and are recomputed on the cloud, so they add no bytes.

3.7. Success criterion

Before running experiments we fixed a target: an operating point is acceptable (*GO*) if it reaches at least 95 % of always-cloud accuracy (76.17 % top-1) while escalating at most 30 % of images.

4. Embedded system implementation

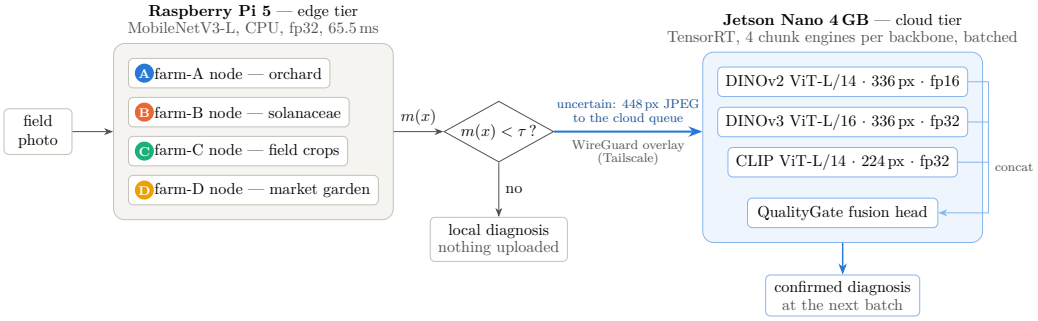


Figure 1: System overview: selective edge–cloud escalation.

4.1. System architecture

Figure 1 summarises how one image travels through the system: each farm node runs the edge model on every image, answers the farmer without waiting for the ensemble and computes the margin $m(x)$ between its two most probable classes; only uncertain images ($m < \tau$) are uploaded, resized to a 448 px shorter side, to the cloud node's queue, which is processed in batches, backbone by backbone with one chunk engine in memory at a time (Section 5.4), and the ensemble's diagnosis is returned. All traffic travels over an encrypted WireGuard overlay. Figure 2 shows the system as deployed. It

has three tiers, each a set of independent services that communicate only through small HTTP/JSON interfaces. The *edge tier* runs one service per farm (`plant-edge-farm@A...D`, managed by systemd on the Raspberry Pi 5); the *cloud tier* runs a single service (`plant-cloud-trt` on the Jetson Nano); and an *operations tier* on a workstation drives the experiment, records every event and displays the fleet's state. Four paths connect them. A capture is sent to its farm's edge service (path 1), which answers with its own diagnosis, margin and upload size (path 2). Only if the margin is below τ does the edge service also resize the image and place it on the cloud's queue (path 3), before answering; the answer therefore waits for the upload to be acknowledged but never for the cloud's diagnosis. Twice a day the queue is processed as one batch and the ensemble's confirmed diagnoses are returned (path 4). Because the tiers share nothing but these interfaces, a farm can be added by starting one more edge service, and the cloud tier can move from the Nano to any larger machine without changing the edge.

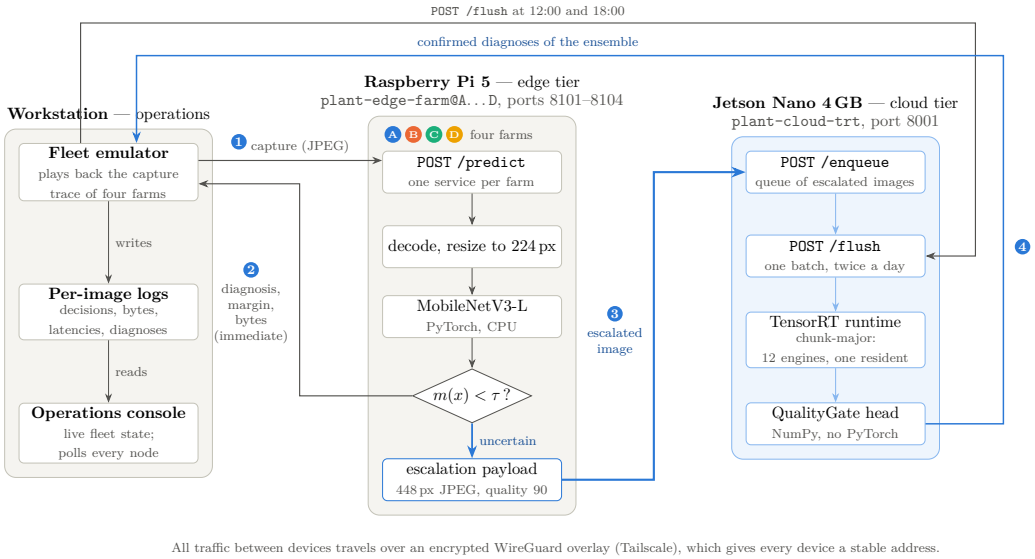


Figure 2: System architecture as deployed.

4.2. Hardware and software

Table 1 summarises the testbed. The edge is a Raspberry Pi 5 running the edge model in PyTorch on its CPU. The cloud node is a 4 GB Jetson Nano, a low-cost embedded GPU module, chosen deliberately: if a three-backbone

foundation ensemble can be served from a device of this class, the "cloud" in our architecture can be a cooperative- or farm-level box rather than a data centre. The nodes communicate over a WireGuard-based overlay network [38] (Tailscale), which encrypts all traffic and gives each device a stable address regardless of its physical network. Both servers expose a small HTTP/JSON interface in which images travel as base64-encoded JPEG. All byte counts in this paper are those of the JPEG payload; the base64 text encoding of our prototype adds a third on top of them, an overhead that a binary upload removes and that leaves every reduction factor unchanged.

Table 1: Embedded testbed.

	Edge	**Cloud**
Device	Raspberry Pi 5 Model B	Jetson Nano Developer Kit
Processor	4× Cortex-A76, 2.4 GHz	4× Cortex-A57 + 128-core Maxwell GPU
Memory	16 GB	4 GB (shared CPU/GPU)
OS	Debian 13	Ubuntu 18.04 (JetPack 4.6.1, L4T R32.7.1)
Runtime	Python 3.13, PyTorch 2.6 (CPU)	Python 3.6, TensorRT 8.2.1, CUDA 10.2
Model	MobileNetV3-L, fp32	DINOv2-L (fp16), DINOv3-L and CLIP-L (fp32); 4 chunk engines each
Power mode	default (`ondemand`)	MAXN

4.3. Edge node

Each edge node is a stateless HTTP service. For every image it decodes the JPEG, applies the edge model, computes the margin and, if $m(x) < \tau$, forwards the image to the cloud node; the uploaded byte count is recorded per image once the cloud acknowledges it. Escalation is *deferred*: the node uploads the image to the cloud node's queue and returns its own label as soon as the upload is acknowledged, so the farmer never waits for the ensemble's diagnosis, which follows when the queue is processed (Section 5.4). The upload itself, however, is on the answer path of our prototype: it adds 69 ms (median) on the testbed's local network, about 0.5 s for a median payload on a 1 Mbit/s rural uplink, and up to the 60 s request timeout if the cloud node cannot be reached, after which the node answers with its own label.

All latencies reported in this paper include this upload. A production node should upload from a background queue so that the answer never waits for the network; the released edge server offers this as an option, which was added after the measurements and is not evaluated here.

Escalation payload. Field photographs are often far larger than the cloud model needs: some PlantWild images exceed 12 megapixels, while the cloud backbones operate at 224–336 px. Uploading at the camera's resolution therefore wastes most of the escalated bytes. The edge instead resizes an escalated image so that its shorter side is 448 px and encodes it as JPEG at quality 90. The policy is chosen on the validation split, so that no test image informs a design choice: Table 2 shows the trade-off on its 1 820 images, measured with the reference cloud pipeline. Agreement is the fraction of images for which the cloud's diagnosis from the payload equals its diagnosis from the original file, and KB are mean sizes over the same images (the original files average 148.9 KB on disk). Among the downscaled payloads, 448 px at quality 90 keeps the highest agreement (98.4 %) with no loss of cloud accuracy, for 2.3 × fewer bytes than the full-resolution image at the same quality; a 336 px payload would save another 30 % of the bytes at a small cost in agreement and accuracy. The always-cloud baseline in the deployment uses the identical payload, so the measured reduction factor isolates the effect of selective escalation.

Table 2: Escalation payload policy (validation split).

Payload	**KB / escalation**	**Agreement**	**Cloud top-1**
Original file	148.9	—	79.6 %
Full resolution, JPEG q90	133.2	99.2 %	79.8 %
Shorter side 448 px, q90	**59.1**	**98.4 %**	**79.8 %**
Shorter side 336 px, q95	57.1	98.1 %	79.6 %
Shorter side 336 px, q90	41.9	97.8 %	79.3 %
Shorter side 224 px, q90	22.9	95.4 %	78.8 %

If the cloud is unreachable, the node falls back to its own prediction and records the failure, so that an outage degrades accuracy rather than availability. The deployment uses the validation-calibrated $\tau = 0.138$ (Section 3.4).

4.4. Trace-driven emulation of a multi-farm week

A field deployment spans days, with hours between captures. To study that regime within an experimental session we use trace-driven, hardware-in-the-loop emulation rather than simulation: a recorded workload (the test images, arranged into a week of captures) is played back through the real devices and the real network, so that every inference, every transfer and every byte is executed and measured rather than modelled. Only the clock is compressed — idle time between captures is skipped — and we refer to its time axis as simulated hours and days. We call this experiment an *emulated deployment*, or the *emulation* for short. It was run twice: the *baseline deployment* uses the original edge model, and the *distilled deployment* (Section 6.7) the edge model distilled from the ensemble.

Virtual farms. Four farm nodes run as independent edge processes on the Raspberry Pi 5, each with its own identifier, port, routing decisions and connection to the cloud node; the cloud therefore serves four distinct clients. Farms differ in what they grow: the 89 classes are grouped by crop into an orchard (apple, banana, blueberry, cherry, citrus, coffee, grape, maple, peach, plum, raspberry, strawberry; 1 360 test images), a solanaceous-crop farm (bell pepper, eggplant, potato, tobacco, tomato; 701), a field-crop farm (bean, corn, garlic, ginger, rice, soybean; 707) and a market garden (basil, broccoli, cabbage, carrot, cauliflower, celery, cucumber, lettuce, squash, zucchini; 909). Together they cover the full 3 677-image test split exactly once.

Schedule. Each farm's images are spread evenly over the daylight hours (07:00–19:00) of seven simulated days, giving 84 capture hours per farm; within an hour, requests from different farms are interleaved. One simulated hour lasts at least one second of wall-clock time and is stretched when its captures take longer, so the emulation never drops or overlaps work.

What is and is not emulated. Requests are issued one at a time, so every edge latency is that of a single request on an otherwise idle device — exactly what a farm owning its own Pi would observe. Upload sizes, escalation decisions, diagnoses and per-stage latencies (edge inference, network, cloud inference) are measured for every image. What the emulation compresses is idle time; what it emulates is the multiplicity of edge devices. Table 3 lists the provenance of every quantity reported in this paper.

Table 3: Provenance of the reported quantities.

Quantity	**Provenance**
Accuracy, escalation rate (offline)	measured on the PlantWild test split
Bandwidth reduction factor $1/r$	measured (independent of image size)
Absolute KB per image (offline)	escalation rate $\times$ measured mean payload S
Absolute KB per image (deployment)	measured JPEG bytes of every upload
Edge latency	measured on the Raspberry Pi 5
Cloud latency	measured on the Jetson Nano
Cloud memory and energy	measured on the Jetson Nano (`tegrastats`: RAM, board input power)
Edge energy	measured on the Raspberry Pi 5 (power-management IC: SoC and memory rails, a lower bound)
Distillation and loop pilot	offline, on the workstation; test split as in Section 3.1
Network latency	measured over the overlay; local network, i.e. a best case for rural links
Four farms	four processes on one physical Pi (emulated multiplicity)
Seven days	simulated clock; idle time compressed, work not compressed

5. Running a foundation-model ensemble on end-of-life embedded hardware

The Jetson Nano is typical of hardware that farms, cooperatives and agricultural schools already own or can buy cheaply, but that the software ecosystem has left behind. NVIDIA's last software release for it (JetPack 4.6) ships Ubuntu 18.04, Python 3.6, CUDA 10.2 and TensorRT 8.2, whereas the reference cloud implementation relies on PyTorch 2, `timm` and `open_clip`, none of which runs on that stack; DINOv3 in particular requires a far newer toolchain. Memory is the second wall: the board's 4 GB are shared by CPU and GPU, while the fp32 weights of the three backbones alone occupy about 3.4 GB. Rather than rebuild a framework stack on the device, we treated the port as an engineering problem and solved it with five principles, each of which transfers to other legacy accelerators. Table 4 summarises the decisive

328 obstacles; Appendix A lists all of the adaptations the port required.

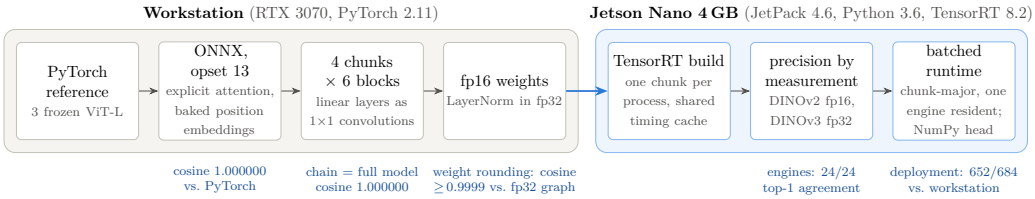


Figure 3: Porting the cloud ensemble to the Jetson Nano.

Table 4: Decisive obstacles of the port, their causes and the adaptations that resolved them.

Obstacle	**Cause**	**Adaptation**	**Evidence**
Reference server cannot run	JetPack 4: Python 3.6, CUDA 10.2	ONNX export on the workstation; torch-free NumPy server	graphs cosine 1.000000; preprocessing bit-identical
Operators rejected by TensorRT 8.2	fused attention, cubic resize, shape-dependent position embeddings	explicit attention; position embeddings baked as constants; constant folding	no unsupported operator left
A whole ViT-L cannot be compiled	ONNX parser alone peaks at 2.75 GB	fp16 weights; four chunks of six blocks	chained chunks cosine 1.000000
969 MB of device memory per chunk	weights fused into one opaque kernel graph	every linear layer as a 1×1 convolution	engine 612 → 157 MB
NaN features from DINOv3	residual stream 1.6×10^5, above the fp16 range	precision chosen per backbone from measured ranges	DINOv3 engine cosine 1.0000
Only 9 of 12 chunks fit	≈0.35 GB per loaded fp16 chunk; fp32 chunks twice as large	chunk-major batched inference, deferred escalation	peak 2.32 GB of 3.86 GB
Residual on-device deviation	Pillow 8.4 links a different JPEG decoder	isolated (decoding, not resampling) and measured; accepted with stock JetPack 4 packages	652/684 agreement in deployment

5.1. Decouple the model from its framework

On the workstation, each backbone is exported to ONNX [39] at opset 13, the newest opset that TensorRT 8.2 parses (Fig. 3). The blue labels in the figure give the fidelity check passed at each stage, detailed in Section 5.5. Fused scaled-dot-product attention has no opset-13 equivalent, so attention is traced in its explicit $\mathrm{softmax}(QK^\top/\sqrt{d})V$ form, which is mathematically identical. Operations that depend on the input shape — DINOv2's bicubic resizing of its position grid, DINOv3's rotary position embedding — are evaluated once for the fixed input size and stored as constants, and an export is rejected if any operator unsupported by TensorRT 8.2 remains. Image preprocessing is re-implemented without PyTorch — Pillow decodes the JPEG and performs the bicubic resize, as torchvision does, and NumPy crops, normalises and lays out the tensor — and the QualityGate head of Eq. (1) is re-implemented in NumPy, so the serving process on the Nano needs neither PyTorch nor its dependencies.

5.2. Compile within the memory ceiling

A whole ViT-L cannot be compiled on the Nano: TensorRT's ONNX parser alone peaks at 2.75 GB of RAM for a 0.6 GB fp16 graph, before the builder has started (Fig. 4b). Figure 4b compares the memory each step of the port needs with the board's shared CPU/GPU memory; the grey bars are the steps that did not fit. Each backbone is therefore exported as four consecutive chunks of six transformer blocks (the first also holding the patch embedding, the last the pooling and projection), converted to fp16 weights with LayerNorm arithmetic kept in fp32, and compiled into a TensorRT engine [40] one chunk at a time. Weights are stored in fp16 for all three backbones; the precision in which an engine *computes* is chosen per backbone when it is built (next subsection); at run time the chunks are chained and compute exactly the full forward pass (cosine 1.000000 against the unsplit model). A second, less visible limit followed: TensorRT 8.2 fuses transformer layers whose weights enter through matrix products into one opaque kernel graph, which held 969 MB of device memory for six blocks — about 13 bytes per parameter. Expressing every linear layer as the equivalent 1×1 convolution keeps the weights in TensorRT's native layers and shrinks the engine for the same six blocks from 612 to 157 MB. The residual stream keeps its (batch, tokens, channels) layout: each linear layer reshapes its input to a $1 \times C \times (B\,N) \times 1$ tensor, applies the convolution and reshapes the result

back, and attention is computed on (batch, heads, tokens, head dimension) tensors as before.

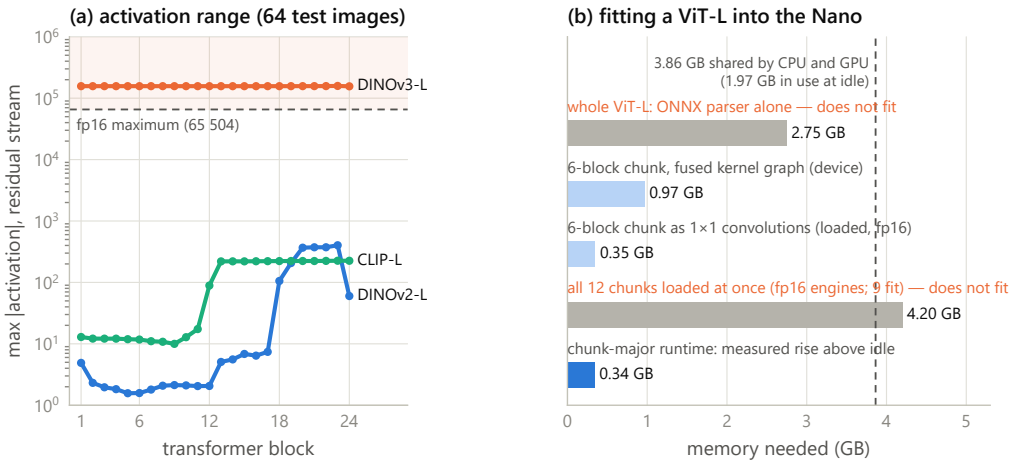


Figure 4: Activation ranges and memory requirements of the port. (a) Largest residual-stream activation per transformer block. (b) Memory needed at each step of the port.

5.3. Choose precision by measurement

Half precision halves weights and load time, but it does not suit every backbone. Our first fp16 build of DINOv3 produced NaN features. The natural suspect, LayerNorm overflowing in fp16, was wrong: pinning LayerNorm, and then every non-convolution layer, to fp32 changed nothing. Profiling the reference model (Fig. 4a) on 64 test images located the cause: DINOv3's residual stream carries activations of up to 158 259 from its first block onwards, beyond the largest fp16 value (65 504), whereas DINOv2 and CLIP peak at 403 and 225. DINOv2 therefore runs in fp16 and DINOv3 in fp32. Our workstation check of the fp16-weight graphs (Table 5) could not have revealed the overflow: onnxruntime's CPU provider has no fp16 kernels for these operators and computes them in fp32, so the check validated the rounding of the weights, not fp16 arithmetic — run there, the first DINOv3 chunk's hidden state reaches 1.57×10^5 without overflowing. An fp32 engine is built from the same fp16-weight graph with every layer pinned to fp32 by TensorRT's precision constraints: it keeps the rounded weights but computes in fp32, and the rounding is harmless, as the engines reproduce the fp32 reference to a cosine of at least 1.0000. CLIP was moved to fp32 as well while we

investigated a deviation that we later traced to the image library rather than to precision (Section 5.5). A later check confirmed that fp32 is unnecessary for CLIP: fp16 CLIP engines reproduce the fp32 ones to a cosine of at least 0.9998 on the 24 reference images, with the same pipeline agreement, while cutting CLIP's compute from 1 613 to 1 158 ms per image and its loading from 19 to 10 s. The baseline deployment used fp32 CLIP; fp16 is the better choice for new deployments.

5.4. Run within the memory ceiling: chunk-major batched inference

Even in compact form, a loaded fp16 chunk engine occupies about 0.35 GB of the Nano's shared memory (157 MB of weights on the device plus the runtime's host-side copy and scratch), and we measured, with fp16 engines for all three backbones, that at most nine of the twelve chunks can be resident at once, even with a single activation buffer shared by all of them. The deployed mix is larger still: an fp32 chunk engine occupies 294 MB on disk against 147 MB for an fp16 one, and the twelve deployed engines total 2.87 GB. The full ensemble therefore cannot answer requests synchronously on this device. We resolve the conflict with a design that matches how diagnoses are used on farms: an escalated image is *queued* on the cloud node, and the edge returns its own label to the farmer as soon as the image is queued. The cloud node processes its queue in batches, chunk by chunk: it loads the first chunk of DINOv2, passes every queued image through it, frees it, loads the next chunk, and so on through DINOv3 and CLIP, keeping each image's intermediate hidden state (up to 2.4 MB for DINOv2's 577 tokens) in host memory, and finally applies the head. With its decoded payload and preprocessed input, a queued image holds about 4.5 MB during a batch, so the released server processes queues of more than 128 images in sub-batches of 128 (about 0.6 GB), reloading the engines for each; this cap was added after the measurements, in which no batch exceeded 61 images. At no time is more than one chunk engine resident. In the emulation the queue is processed twice a day, after the 12:00 and 18:00 captures, so every escalated image receives the ensemble's diagnosis on the day it was taken; the batch interval trades confirmation delay for model-loading overhead. The server also implements a synchronous mode with all engines resident, intended for cloud nodes with more memory; we did not test it.

To quantify the design on the device, we processed 2 batches of 48 escalated images — the median batch of the baseline deployment, with the same payload — while sampling RAM and the board's input power rail

with `tegrastats` about eight times per second (Fig. 5). In the figure, shading marks the backbone being processed; the twelve dips in power are the twelve chunk-engine loads, between which the GPU passes all queued images through the resident chunk. The dashed lines mark the board's total shared memory and its idle power draw. Memory use never exceeds 2.32 GB of the 3.86 GB (1.97 GB at idle, including the operating system and the idle server): processing a batch adds about 0.34 GB on average, the footprint of one chunk engine — up to 0.46 GB while an fp32 engine (DINOv3, CLIP) is resident and 0.22 GB for the fp16 DINOv2. The board draws 6.3 W on average while processing a batch (1.2 W at idle, 7.9 W at most), and a batch takes 6.1 min. Each escalated image costs 49 J of board energy including its share of model loading, or 40 J above the idle draw — about 14 Wh per thousand escalated images.



Figure 5: Jetson Nano memory and board power during one batch of 48 escalated images.

5.5. Verify every stage

A port is only useful if it computes the same function, so we verify each stage against the reference pipeline (Table 5). Workstation checks compare against the PyTorch reference; checks on the Nano compare TensorRT outputs with reference features computed on the workstation. The exported graphs match PyTorch to a cosine similarity of 1.000000, the NumPy preprocessing reproduces the reference tensors exactly, and the NumPy head

matches the original to within 3×10^{-6} in the logits. The reassembled pipeline agrees with the offline reference cloud predictions on 299 of 300 test images (99.7 %). On the Nano, fed the workstation's preprocessed tensors, the TensorRT engines reproduce the reference features to a cosine of 1.0000 (DINOv3, fp32), 1.0000 (CLIP, fp32) and 0.9965 (DINOv2, fp16) and the reference top-1 prediction on 24/24 images. The complete on-device pipeline agrees on 23/24: the residual deviation (minimum cosine 0.9474) does not come from TensorRT but from the image library — Pillow 8.4, the last release for JetPack 4's Python 3.6, links a different JPEG decoder (IJG libjpeg 9) from the workstation's Pillow 12 (libjpeg-turbo 3.1). On the 24 reference images the two decoders return different values for 36 % of pixels, by up to 67/255, so the preprocessed tensors differ by up to 0.59 normalised units; resampling is not involved, as both versions resize identical decoded pixels bit-identically. Over the baseline deployment, the Nano's diagnosis matches the workstation's offline prediction for 652/684 escalated images. Isolating causes in this way — engines fed identical tensors, then the complete pipeline — is what separated a precision problem that had to be corrected (DINOv3) from a library difference that we accepted: removing it would require a matched JPEG decoder on both machines, which stock JetPack 4 packages do not provide.

6. Results

6.1. Edge capacity determines the operating point

Table 6 and Fig. 6 compare the three edge models. For each model we report the smallest escalation rate at which margin escalation reaches 95 % of always-cloud accuracy (76.17 %), denoted escalation @95 %. In Fig. 6a, dots mark where each model first reaches this target; Fig. 6b plots edge compute against the escalation each model needs, with bubble area proportional to the parameter count, and the dotted lines mark the 30 % escalation budget. Escalation @95 % is computed exactly, image by image, by escalating images in increasing order of margin until the target is reached. With MobileNetV3-Small as the edge (61.08 % top-1), the margin rule reaches the 95 % target only at 43.5 % escalation, failing the pre-specified criterion. This failure is not a weakness of the margin rule: a learned router trained on the same edge outputs gained only two points (41.3 % escalation), and adding 1 024-dimensional DINOv2 image features to the router lowered its AUC from 0.785 to 0.768 (42.0 % escalation), a sign of overfitting the 1 820 validation images.

Table 5: Fidelity of the embedded cloud pipeline, stage by stage.

Stage	**Check**	**Result**
ONNX export (3 backbones)	cosine vs. PyTorch, min over images	1.000000
Chunking + 1×1-convolution form	chained chunks vs. unsplit model, cosine (PyTorch and ONNX)	1.000000
fp16 weights, fp32 LayerNorm	cosine vs. fp32 graph, min (computed in fp32: tests weight rounding)	≥ 0.9999
Pillow + NumPy preprocessing	max. abs. difference vs. torchvision	0
NumPy QualityGate head	max. abs. logit difference	2.9×10^{-6}
Full pipeline	agreement with offline cloud predictions	299/300 (99.7 %)
Edge node (Raspberry Pi 5)	label agreement with offline edge predictions	300/300
	margin, mean / max. abs. difference	$8{\times}10^{-4}$ / $7.8{\times}10^{-3}$
Nano engines, workstation-preprocessed inputs		
TensorRT fp16, DINOv2-L	cosine vs. reference, min / mean	0.9965 / 0.9979
TensorRT fp32, DINOv3-L	cosine vs. reference, min / mean	1.0000 / 1.0000
TensorRT fp32, CLIP-L	cosine vs. reference, min / mean	1.0000 / 1.0000
Engines + head	top-1 agreement with reference	24/24
Complete on-device pipeline (Nano image library)		
DINOv2-L / DINOv3-L / CLIP-L	cosine vs. reference, min	0.9955 / 0.9874 / 0.9474
Full pipeline (Nano)	top-1 agreement with reference	23/24
Full pipeline (Nano, deployment)	agreement with offline cloud predictions, every escalated image	652/684

For this edge model, every edge-side signal we tried saturated at an AUC of 0.76–0.79 for predicting useful escalations — the information an edge-side router can see is bounded by the edge model itself.

Replacing the edge model changes the picture. MobileNetV3-Large raises edge accuracy by 8.2 points to 69.32 % and halves the required escalation to 20.3 %, comfortably passing the criterion. EfficientNet-B0 reaches the same edge accuracy (69.35 %) but needs slightly more escalation (21.4 %) and 1.8× the multiply–accumulate operations, so MobileNetV3-Large is our edge model for the remainder of the paper.

Table 6: Edge models on the PlantWild test split.

Edge model	**Params**	**GMACs**	**Top-1**	**Esc. @95 %**	**Criterion**
MobileNetV3-Small	1.61 M	0.059	61.08 %	43.5 %	fail
MobileNetV3-Large	**4.32 M**	**0.225**	**69.32 %**	**20.3 %**	**pass**
EfficientNet-B0	4.12 M	0.400	69.35 %	21.4 %	pass

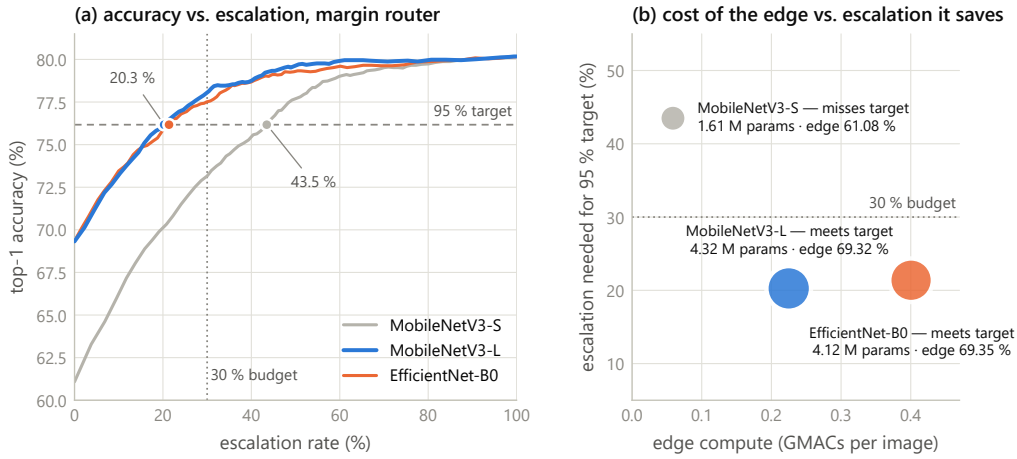


Figure 6: Edge capacity and the escalation it requires. (a) Accuracy versus escalation rate. (b) Edge compute versus escalation needed for the 95 % target.

The oracle curve in Fig. 7 and the last column of Table 8 add a further observation. At 20 % escalation the oracle reaches 84.80 % with MobileNetV3-Large, *above* the always-cloud model: the edge and the cloud make partly different errors, so a perfect router would beat both. The gap between the

practical rules and the oracle measures how much a better uncertainty signal could still gain; it does not depend on the cloud model, which is unchanged.

6.2. Accuracy–bandwidth trade-off

Table 7 and Fig. 7 place the operating points on the bandwidth axis. Absolute traffic uses the measured mean payload $S = 60.2\,\mathrm{KB}$ (Section 3.5); the reduction factor does not depend on S. In the figure, the ring marks the operating point with the validation-calibrated threshold, the shaded region the pre-specified success criterion, and the light curve the oracle router, an upper bound for any escalation rule; the lower block of the table reads both routers' test curves at the same accuracy, for comparison only. With the validation-calibrated threshold, margin escalation from MobileNetV3-Large reaches 75.82 % top-1 on the test split — 94.6 % of always-cloud accuracy — while escalating 18.6 % of images: 11.2 KB per diagnosis instead of 60.2 KB, a 5.38-fold reduction. On the validation split, where τ was set, the rule meets the criterion by construction (95.0 % of cloud accuracy at 19.3 % escalation); the 0.4-point shortfall on test is the generalisation gap of a threshold fixed in advance. Its size is what the calibration's sampling variability predicts: recalibrating τ on 2 000 bootstrap resamples of the validation split gives a test accuracy between 92.1 and 96.0 % of cloud accuracy (5th–95th percentile; median 94.6 %) and an escalation rate between 12.3 and 23.5 %. A deployment that must meet the target with high confidence can take the 95th percentile of the bootstrap thresholds instead ($\tau = 0.198$): it then delivers 96.0 % of cloud accuracy on test at 23.5 % escalation, still a 4.3-fold reduction. Read directly off the test curve, the 95 % line is crossed at 20.3 % escalation (12.2 KB). Measured by macro-F1, which weights the 89 classes equally, the operating point delivers 72.9 % against 65.8 % for the edge alone and 77.5 % for always-cloud, the same ordering as top-1. The curve is markedly concave: the first escalated images are the most valuable. Escalating 30 % of images already closes 80 % of the gap between edge and cloud accuracy (Table 8), so the last fifth of that gap costs the remaining 70 % of the traffic. Table 8 shows that the three scalar signals are within 0.5 points of each other at every budget; we adopt the margin because it ties for best at the target and is the simplest to interpret.

Figure 8 follows every test image through the system. Band widths are proportional to image counts, and escalated images are drawn in blue. Of the 684 escalated images, 468 (68 %) were edge errors, so the margin spends uplink overwhelmingly where it is needed; the cloud rescues 289 of them and

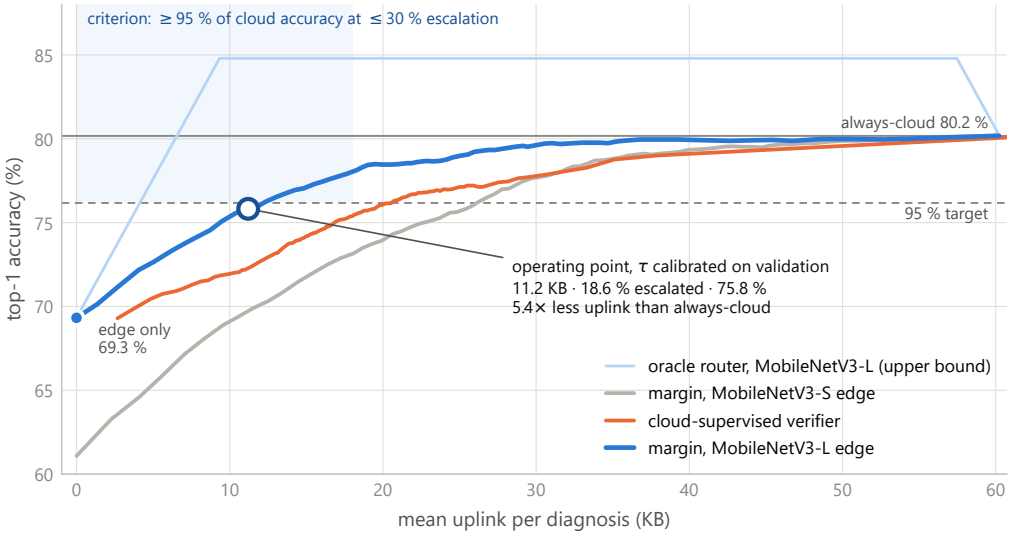


Figure 7: Accuracy versus mean uplink per diagnosis (offline, test split).

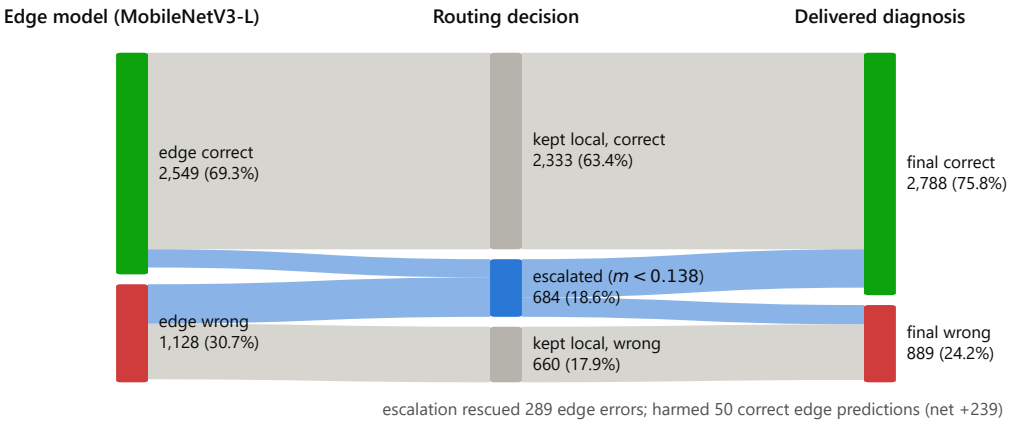


Figure 8: Flow of the 3 677 test images at the calibrated threshold.

Table 7: Operating points on the PlantWild test split (offline).

Operating point	**KB/img**	**Escal.**	**Top-1**	**% cloud**
Edge only (MobileNetV3-L)	0.0	0 %	69.32 %	86.5 %
Margin escalation, τ=0.138 **calibrated on validation**	**11.2**	**18.6 %**	**75.82 %**	**94.6 %**
Always-cloud	60.2	100 %	80.17 %	100 %
Test curves read at 95 % of cloud accuracy				
Margin escalation	12.2	20.3 %	76.18 %	95.0 %
Cloud-supervised verifier	20.2	29.0 %	76.18 %	95.0 %

overturns 50 correct edge predictions, a net gain of 239 diagnoses. The figure also shows the limit of any edge-side signal: 660 edge errors are made with a confident margin and never leave the farm.

The rule also adapts across crops without any per-crop tuning (Fig. 9). The figure shows, per crop, accuracy before and after escalation and the share of images escalated; the letters give each crop's farm in the emulation (Section 4.4). Escalation ranges from 3 % of raspberry images to 36 % of bean images, and it is strongly anti-correlated with the edge model's accuracy on the crop (Pearson $r = -0.85$): bandwidth flows to the crops the edge finds hardest, which are also those that gain most — soybean (+16.7 points), peach (+14.1), zucchini (+13.9) and bean (+12.9).

Table 8: Top-1 accuracy of MobileNetV3-L escalation at fixed escalation budgets, by signal.

Escalation	**Max. probability**	**Margin**	**Entropy**	**Oracle**
5 %	71.72	71.34	71.50	74.33
10 %	73.43	73.27	73.21	79.33
15 %	74.79	74.82	74.84	84.34
20 %	76.09	76.09	75.80	84.80
25 %	77.21	77.10	76.67	84.80
30 %	78.00	78.00	77.45	84.80

6.3. A cloud-supervised verifier does not pay for itself

If edge-side signals are bounded by what the edge knows, a router that is trained with the cloud's answers might do better. It does not (Table 9,

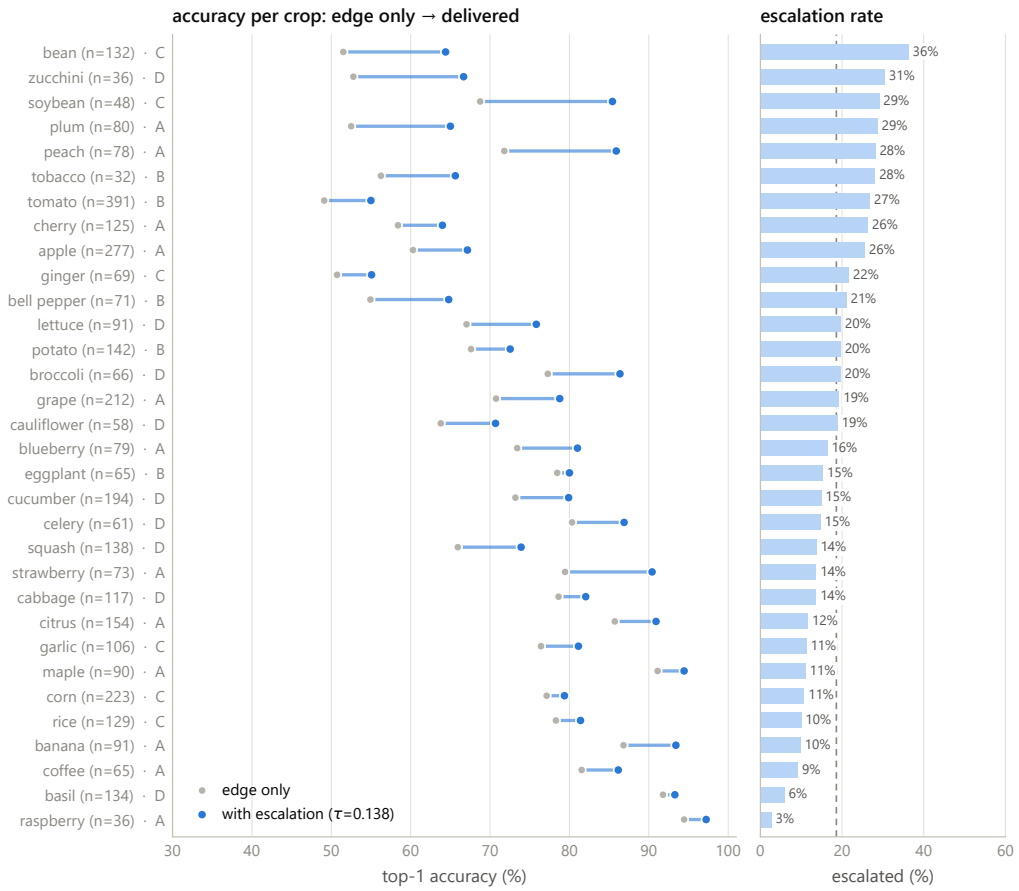


Figure 9: Per-crop accuracy before and after escalation, and per-crop escalation rate.

Fig. 10). The best verifier scores an AUC of 0.731 for identifying useful escalations, below the scalar $1 - m(x)$ at 0.810. On the bandwidth axis the verifier is dominated twice over: it needs more escalations for the same accuracy (29.0 % versus 20.3 %) and it adds its 2.67 KB message to every image. From Eq. (3), merely breaking even would require the verifier to escalate $M/S \approx 4.4$ points *fewer* images than the margin rule; it escalates 8.8 points more. The tax grows as payloads shrink: the more efficiently escalated images are encoded, the larger an always-on message is relative to them. The verifier's ranking also generalises poorly: the best target reaches an AUC

of 0.848 on its held-out validation images but 0.731 on test, the signature of a 1 372-dimensional input fitted on 1 820 images. On this evidence the embedding carries no routing information beyond what the edge posterior already exposes for free.

Could the scalar signals themselves be improved? The edge models are trained with focal loss and label smoothing, which change the scale of their softmax, so we also calibrated the posterior by temperature scaling [32] on the validation split ($T = 0.84$; expected calibration error on test 0.097 before and 0.054 after) and tried the margin between the two largest logits. Every variant is calibrated by the same rule and evaluated on test (Table 9). For the original edge model, none improves on the plain margin by more than a few tenths of a point of escalation; for the distilled model of Section 6.4, the maximum probability needs 1.4 points less escalation than the margin. The differences between scalar signals are small compared with those between edge models, which is why we keep the pre-specified margin rule.

Table 9: Escalation signals: ranking quality and escalation needed.

	Original edge		**Distilled edge**	
Signal	**AUC**	**Esc. @95 %**	**AUC**	**Esc. @95 %**
1− margin (deployed)	0.810	20.3 %	0.830	15.9 %
temperature-scaled	0.811	20.7 %	0.829	16.6 %
1− maximum probability	0.806	20.4 %	0.823	14.5 %
temperature-scaled	0.810	20.2 %	0.829	15.6 %
Entropy	0.790	22.7 %	0.797	15.7 %
temperature-scaled	0.797	21.2 %	0.814	14.8 %
−(logit margin)	0.810	21.5 %	0.821	17.3 %
Cloud-supervised MLP verifier	0.731	29.0 %	—	—

6.4. The ensemble as a teacher for the edge model

Section 6.1 showed that edge capacity sets the operating point. A more capable edge model need not be a larger one: the cloud ensemble can teach it. We tested this in two ways (Table 10). Both edge models are evaluated from the same per-image predictions as Table 7, with thresholds calibrated on validation by the rule of Section 3.4.

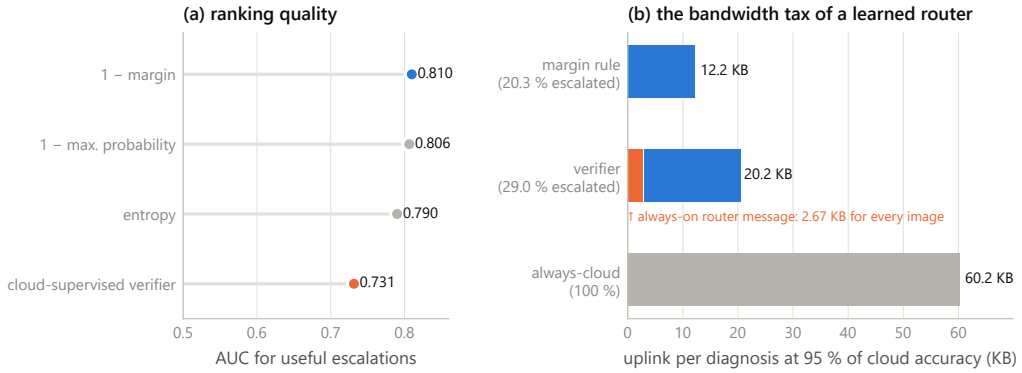


Figure 10: Cloud-supervised verifier versus scalar signals. (a) Ranking quality (AUC). (b) Uplink per diagnosis at 95 % of cloud accuracy.

Distillation before deployment. We retrained MobileNetV3-Large with the recipe and split of Section 3.3, adding to the loss the Kullback–Leibler divergence to the ensemble's class probabilities on each training image [41] (temperature 2, weight 0.5). The probabilities come from the backbone features already computed for the training split, so the teacher adds no inference cost, and the distilled model keeps the architecture and on-device latency of the original. On its own it reaches 71.9 % top-1, against 69.3 % before distillation. With its threshold recalibrated on validation by the same rule (Section 3.4; $\tau = 0.072$), it escalates 12.3 % of test images instead of 18.6 % while delivering 94.0 % of cloud accuracy, so uplink traffic falls 8.1-fold instead of 5.4-fold; on the test curve the 95 % target is reached at 15.9 % escalation instead of 20.3 %. Part of the recalibrated model's saving comes from accepting slightly lower accuracy (94.0 % of cloud accuracy against 94.6 %), but most of it does not: to match the original model's 75.82 %, the distilled model needs 14.3 % escalation instead of 18.6 %, a 7.0-fold reduction instead of 5.4-fold. The gain costs nothing at run time and needs nothing the system does not already have: a stronger edge model obtained from the cloud model itself. Section 6.7 confirms it on the devices.

Closing the loop after deployment. Every escalated image returns with the ensemble's verdict, a training signal the farm receives for free. To test whether it can keep improving the deployed edge model, we split the test set per class into a first "week" of about 1 812 images, whose 339 escalated images are used to fine-tune the deployed model, and a second week of the remaining

images for evaluation, and repeated the experiment over 5 random partitions. Fine-tuning runs for eight short epochs (AdamW, learning rate 10^{-4} with 10 % warm-up and cosine decay, the loss of the distillation recipe), each mixing the escalated images, repeated four times, with 2 000 rehearsal images drawn from the training split by the original training's class-balanced sampler; the threshold stays at the deployed τ. Because further fine-tuning perturbs a trained model by itself, the reference for the escalated images is a control that fine-tunes on rehearsal images only: it needs $+1.5 \pm 0.5$ points more escalation than the unchanged model, consistently across partitions. Against this control, one week of escalations did not improve the edge model, whatever labels the escalated images carried (Table 10; differences in escalation @95 %, mean ± standard deviation over partitions): $+1.4 \pm 1.9$ points with the ensemble's top-1 labels, $+1.7 \pm 2.3$ with its probabilities, -0.3 ± 1.6 with its probabilities on confident escalations only, and $+2.1 \pm 3.1$ even with farmer-confirmed true labels. The fixed-threshold columns show the mechanism: the models fine-tuned on escalations escalate fewer images at τ than the control but deliver lower accuracy, so they become more confident on the hard cases they were shown without becoming more often right. The 339 images — about four per class, and the hardest the edge sees — are memorised rather than generalised from by a model trained on 11 778 images of the same distribution. The value of the loop therefore lies in accumulating escalations over many weeks and in adapting to farms whose images differ from the training data, which an in-distribution benchmark cannot show; distilling the ensemble before deployment is, by contrast, an immediate gain.

6.5. On-device latency and energy

Table 11 reports latencies measured on the devices. Each measurement is for a single image (batch size 1); deployment values are medians over the baseline deployment. On the Raspberry Pi 5, MobileNetV3-Large runs in 65.5 ms per image (fp32, 224×224, mean of 200 runs after warm-up, no thermal throttling). A FLOPs-based projection from the device's nominal throughput had predicted 32.1 ms; the measured figure is about twice as slow, which illustrates why projected latencies should not stand in for measurements on embedded targets. On the Jetson Nano the three backbones take 2 592, 2 673 and 1 613 ms, so an escalated image costs 6.9 s of cloud computation including preprocessing and the head; loading the three backbones adds 36 s per batch, shared by all images in it. Because escalation is deferred, none of this is on the farmer's path to a diagnosis: the farm answers in a median

Table 10: The ensemble as a teacher: distillation and one-week loop pilot.

Edge model	**Edge top-1**	**Escal.**	**Top-1 (% cloud)**	**Uplink reduction**	**Escal. @95 %**
Original MobileNetV3-L	69.3 %	18.6 %	75.8 % (94.6)	5.4	20.3 %
Distilled from the ensemble	**71.9 %**	**12.3 %**	**75.4 % (94.0)**	**8.1**	**15.9 %**
Loop pilot, week 2, mean of 5 partitions; 339 escalations labelled by					
No update	69.4 %	18.4 %	76.0 % (94.9)	5.4	18.7 ± 2.1 %
No new images (control)	69.2 %	20.1 %	76.2 % (95.2)	5.0	20.2 ± 1.9 %
Ensemble, top-1 label	68.9 %	17.1 %	74.9 % (93.6)	5.9	21.7 ± 2.7 %
Ensemble, probabilities	69.3 %	18.6 %	75.4 % (94.2)	5.4	21.9 ± 2.6 %
Ensemble, confident only (131)	69.5 %	18.5 %	75.8 % (94.7)	5.4	19.9 ± 2.3 %
True label (farmer)	68.8 %	17.5 %	74.8 % (93.4)	5.7	22.3 ± 2.5 %

of 77 ms over all images (95 % within 164 ms) — 74 ms for images answered locally and 139 ms for escalated images, whose answer waits for the upload to the cloud queue (Section 4.3) — and the ensemble's confirmation arrives with the next batch. Figure 12a shows this time to the farmer's answer for every image of the baseline deployment: escalated images are answered by the edge as soon as their upload to the cloud queue is acknowledged, and confirmed by the cloud at the next batch. Figure 12b breaks the cost of one escalation into edge inference on the Pi, upload and Nano compute, excluding the wait for the next batch and the per-batch model loading.

Energy was measured on both devices. On the Pi, the power-management IC reports the voltage and current of every supply rail; over 300 edge inferences the SoC and memory rails drew 3.6 W on average (1.7 W at idle), or 0.45 J per image (0.24 J above idle). These rails exclude peripherals powered directly from the 5 V input, so the figure is a lower bound for the board. On the Nano, an escalated image costs 49 J at the board's input (Section 5.4). A diagnosis under selective escalation therefore costs 9.5 J of device energy (every image on the edge, 18.6 % of them also on the cloud node), against 49 J when every image is sent to the same cloud node: 5.1 times less, in line with the bandwidth saving.

6.6. Seven-day, four-farm emulated deployment

Over the emulated week of the baseline deployment, the four farms submitted 3 677 images, and the edge escalated 684 of them (18.6 %) — the

Table 11: Latency measured on the embedded devices.

Device	**Stage**	**Latency**
Raspberry Pi 5	MobileNetV3-L forward (fp32, 224 px), 200 runs	65.5 ms
	FLOPs-based projection, for reference	*32.1 ms*
	edge inference incl. preprocessing (deployment)	70 ms
	answer on the farm, end to end (deployment)	77 ms
	energy per image (SoC and memory rails)	0.45 J
Jetson Nano	DINOv2-L/14, 336 px (4 chunk engines, fp16)	2 592 ms
	DINOv3-L/16, 336 px (4 chunk engines, fp32)	2 673 ms
	CLIP-L/14, 224 px (4 chunk engines, fp32)	1 613 ms
	cloud compute per escalated image (deployment)	6.9 s
	loading the three backbones, per batch (deployment)	36 s
	energy per escalated image (board input)	49 J
Network	escalation upload to the cloud queue (deployment)	69 ms

rate predicted offline for the same images at the same threshold (18.6 %), as expected from the edge node's agreement with the offline model (Table 5). The farms uploaded 40.4 MB in total, against 216.3 MB had every image been sent with the same payload: a 5.4-fold reduction in the bytes actually uploaded (Fig. 11). The figure shows the escalations per simulated hour, by farm, with nights shaded, and the cumulative uploaded bytes against the same images uploaded with the same payload by an always-cloud design; its green curve is the distilled deployment (Section 6.7). The offline model of Eq. (3) predicted 11.2 KB per diagnosis; the devices measured 11.2 KB. The system delivered 76.0 % correct diagnoses, against 69.3 % for the edge alone. On the escalated images, the Nano's ensemble was correct on 67.4 % of them; the workstation's offline predictions for the same images, which like all offline results are computed from the original full-resolution files, reach 66.5 %, and the two agree on 652/684 images (95.3 %). Running the workstation's reference pipeline on both inputs separates the two causes: on these images it is correct on 66.8 % from the original files and on 68.1 % from the 448 px

payload, so the payload itself does not cost accuracy, and the Nano agrees with it on 662/684 images, the remainder being the decoder difference of Section 5.5. The on-device deviation changes individual diagnoses but not accuracy. The cloud node processed 14 batches of 48 images at the median, never received more than 14 escalations in a simulated hour, and confirmed every escalated image on the day it was captured (at most 6 simulated hours later). The whole emulation took 98 minutes of wall-clock time, almost all of it spent on cloud computation.

The farms behave differently (Table 12, Fig. 13): farm B (solanaceae) escalated most often (24 %) and farm D (market garden) least (15 %), in line with the per-farm rates predicted offline from the margins of the same images. A single global threshold therefore translates into farm-specific uplink budgets that follow how hard each farm's crops are for the edge model, which a deployment can monitor and, if needed, tune per farm. Delivered accuracy also differs between farms, from 62.2 % (solanaceae) to 81.6 % (market garden), but mostly because the farms' crops differ in difficulty for the cloud model too: measured against each farm's own always-cloud accuracy (last column of Table 12), every farm receives between 93.8 % and 96.2 % of what the ensemble alone would deliver, solanaceae included (93.8 %). Figure 14 shows the operations console at the end of the baseline deployment. It displays the simulated clock and node status, fleet indicators, escalations per simulated hour, per-farm cards, the topology, latency by stage and a feed of the most recent diagnoses.

Table 12: Per-farm results of the baseline deployment.

Farm	**Images**	**Escal.**	**Upload (MB)**	**Always-cloud (MB)**	**Reduction**	**Top-1**	**Cloud top-1**
A — orchard	1 360	18.9 %	14.6	75.3	5.1×	79.9 %	84.3 %
B — solanaceae	701	23.8 %	9.4	38.9	4.1×	62.2 %	66.3 %
C — field crops	707	17.8 %	7.7	43.6	5.6×	75.1 %	80.1 %
D — market garden	909	14.7 %	8.6	58.6	6.8×	81.6 %	84.8 %
All farms	3 677	18.6 %	40.4	216.3	5.4×	76.0 %	80.2 %

6.7. Distilled configuration on the devices

The ensemble's most effective contribution to the edge is made once, before deployment. Distillation (Section 6.4) leaves the edge model's architecture, size and on-device cost unchanged, so we deployed the distilled

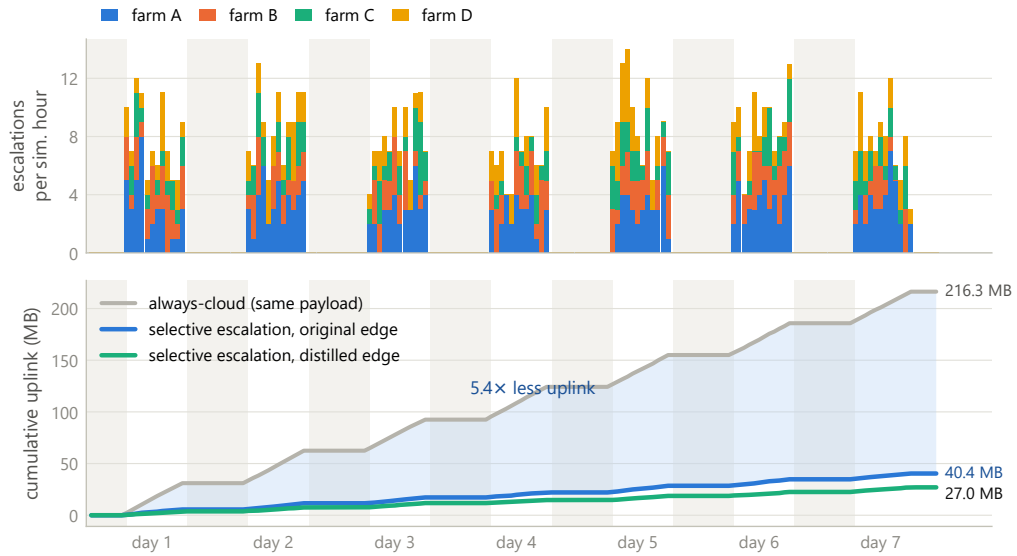


Figure 11: Escalations per simulated hour and cumulative uplink over the seven-day emulated deployments.

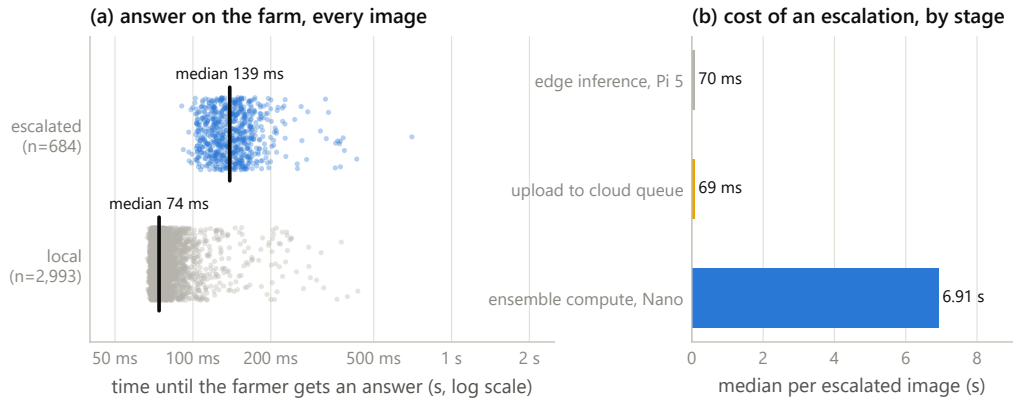


Figure 12: Measured latency during the baseline deployment. (a) Time to the farmer's answer, per image. (b) Median cost of one escalation, by stage.

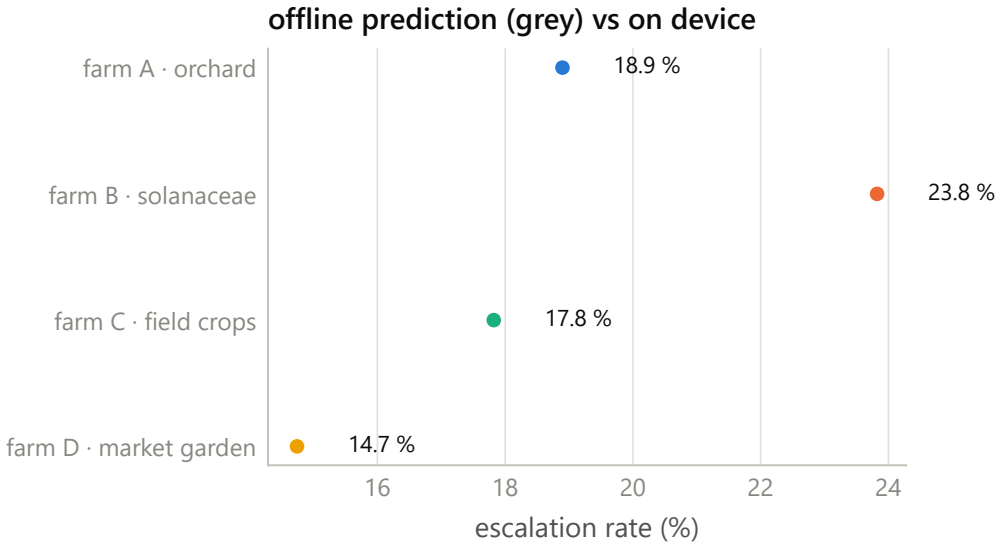


Figure 13: Escalation rate per farm: offline prediction and on-device measurement.

MobileNetV3-Large on the same devices, together with the other improvement found after the baseline deployment, CLIP in fp16 (Section 5); neither changes an interface (Appendix A, item C4). We then repeated the seven-day emulation with identical images, schedule, payload and batch times. The distilled model's threshold was recalibrated on validation by the rule of Section 3.4 ($\tau = 0.072$), and before the run the deployed edge reproduced the distilled model's offline predictions on 300 of 300 test images. Table 13 compares the two deployments.

Bandwidth. The edge escalated 455 images (12.4 %, against the offline prediction of 12.3 %), and the farms uploaded 27.0 MB instead of 40.4 MB: 7.5 KB per diagnosis, an 8.0-fold reduction against always-cloud measured on the devices, compared with 5.4-fold for the original edge model (Fig. 11). Every farm saved more than in the baseline deployment, from 5.3-fold (solanaceae) to 11.3-fold (market garden). Delivered accuracy was 75.3 %, against 76.0 %: both thresholds target 95 % of cloud accuracy on validation, so the stronger edge model spends its gain on bandwidth rather than on accuracy. The gain does not depend on that choice. At equal accuracy the distilled model still escalates far less: offline, it matches the original model's delivered accuracy (75.82 %) at 14.3 % escalation instead of 18.6 %, a 7.0-fold reduction instead

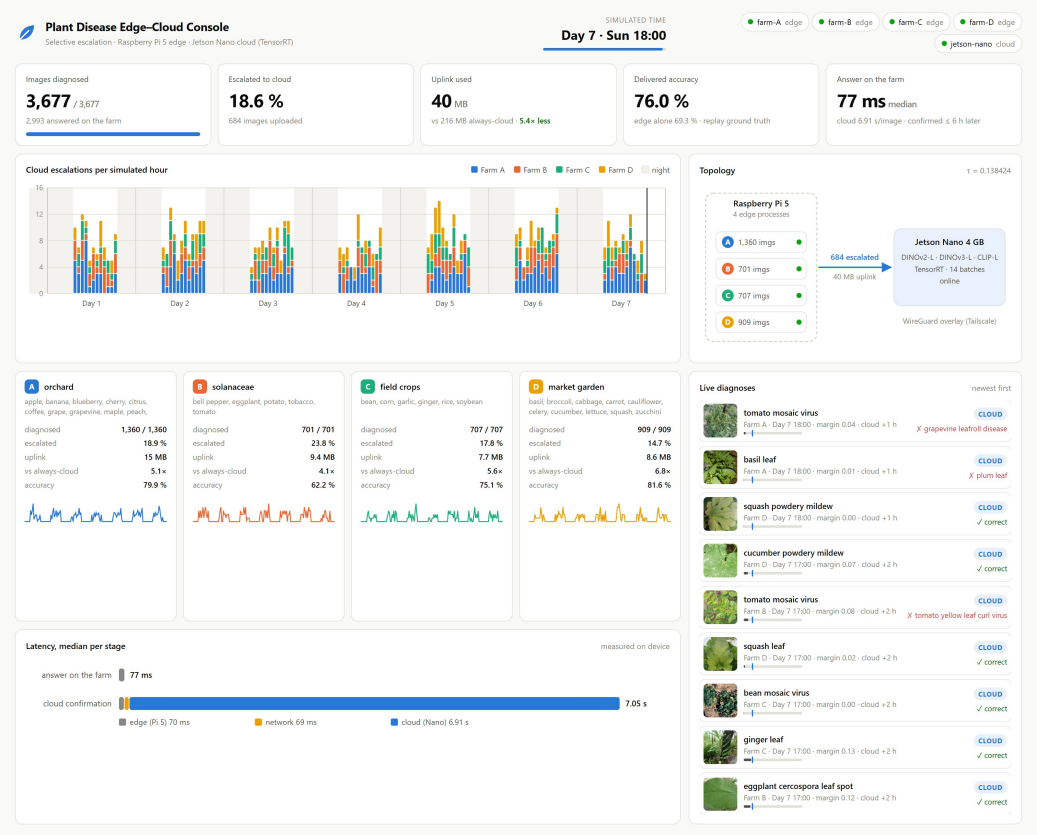


Figure 14: Operations console during the baseline deployment.

of 5.4-fold, and the baseline deployment's 76.0 % at 15.3 % (6.5-fold).

Cost on the devices. Two changes coincide in this deployment, and they act on different quantities. Distillation changes only how many images are escalated; the edge model's latency is unchanged (median answer 76 ms against 77 ms). fp16 CLIP changes only what an escalated image costs on the Nano: compute per escalated image fell from 6.9 to 6.5 s and model loading per batch from 36 to 33 s. The whole emulation took 67 instead of 98 minutes of wall-clock time, a reduction in proportion to the number of escalated images. On the Nano an escalated image costs 49 J with fp32 CLIP and 48 J with fp16 CLIP in batches of 48, and 48 J in batches of 34, the median batch of this deployment (3 runs), so the smaller batches did not raise the energy per image. A diagnosis therefore cost 6.4 J of device energy, against 9.5 J

in the baseline deployment; at the fp32-CLIP energy per escalated image it would have cost 6.5 J, so distillation accounts for nearly all of the saving and fp16 CLIP for the rest. Against an always-cloud design on the same cloud node, which spends 48 J on every image, the distilled configuration uses 7.5 times less device energy per diagnosis. The Nano's diagnoses agreed with the workstation's offline predictions on 429/455 escalated images.

Table 13: The two emulated deployments on the embedded testbed.

	Baseline	**Distilled**
Edge model	original	distilled
CLIP precision on the Nano	fp32	fp16
Threshold τ (calibrated on validation)	0.138	0.072
Images escalated	18.6 %	12.4 %
Uploaded (always-cloud: 216.3 MB)	40.4 MB	27.0 MB
Uplink reduction	5.4×	8.0×
Uplink per diagnosis	11.2 KB	7.5 KB
Delivered top-1	76.0 %	75.3 %
Delivered macro-F1	73.2 %	72.6 %
Edge model alone, top-1	69.3 %	71.9 %
Uplink reduction at equal accuracy (offline)	5.4×	7.0×
Answer on the farm, median	77 ms	76 ms
Nano compute per escalated image, median	6.9 s	6.5 s
Model loading per batch, median	36 s	33 s
Batches (median size)	14 (48)	14 (34)
Nano energy per escalated image	49 J	48 J
Device energy per diagnosis	9.5 J	6.4 J
Wall-clock time of the emulation	98 min	67 min
Nano–workstation agreement, escalated images	652/684	429/455

7. Discussion

Strengthen the edge before engineering the router. The most consequential design decision in our study was not the escalation rule but the edge model.

Moving from MobileNetV3-Small to MobileNetV3-Large cost 2.7× the parameters and 3.8× the operations — still only 65.5 ms on a Raspberry Pi 5 — and halved the escalation rate, whereas a learned edge-side router saved only two points of escalation and a cloud-supervised one did worse than the training-free scalar. Capacity can also be added without changing the architecture: distilling the ensemble into the same MobileNetV3-Large cut the calibrated escalation rate from 18.6 to 12.3 % (Section 6.4), and on the devices from 18.6 % to 12.4 %, an 8.0-fold uplink saving (Section 6.7). Our comparison covers convolutional edge models, which are what runs efficiently on the Pi's CPU with stock PyTorch; lightweight hybrid CNN–transformer models might align better with a transformer teacher, and the same comparison would tell whether they move the operating point further. The reason is informational: an edge-side router can only re-express the edge model's posterior, and the margin already summarises it well. The oracle shows that substantial headroom remains (84.80 % at 20 % escalation), but reaching it requires information the edge does not have; a cheaper path to it is a stronger edge model.

What the bandwidth axis reveals. Latency-centred evaluations treat an always-on router message as negligible; on the bandwidth axis it is a fixed tax that a learned router must earn back through fewer escalations. For field deployments on metered cellular links the relevant figure is the uplink volume per farm and month. At the measured 11.2 KB per diagnosis, a farm capturing 100 images per day would upload 33 MB in a 30-day month, against 177 MB for always-cloud with the same payload, and with the distilled edge model 22 MB; on a 1 Mbit/s uplink an escalated image takes about 0.5 s to transmit.

A cloud that fits on a farm. Serving a 0.9-billion-parameter ensemble from a 4 GB Jetson Nano shows that the "cloud" in selective escalation need not be a data centre. A cooperative, an extension office or a large farm can host the large model locally, which keeps field images within the community's network, removes the dependency on wide-area connectivity for the cloud tier, and turns a recurring service cost into a one-off hardware cost. It does not come for free. Each escalated image costs 6.9 s of Nano compute, and because the full ensemble does not fit in memory, the cloud tier must work in batches; in the baseline deployment a batch of 48 images also paid 36 s to load the three backbones, and the board spends 49 J per escalated image

(Section 5.4). Selective escalation is what makes this affordable: at 18–19 % escalation the Nano sees one image in five, and at the distilled edge model's 12.4 % one image in eight, whereas an always-cloud design would multiply its workload, and its energy use, more than fivefold and eightfold respectively. Beyond this study, the port shows that hardware the software ecosystem has left behind can still serve current foundation models, provided memory, precision and fidelity are engineered explicitly; the same five principles apply to other legacy accelerators that farms and cooperatives already own. The batched design suits scouting and monitoring workflows, in which a confirmed diagnosis within hours is valuable but an instant one is not required; the farmer always receives the edge's answer without waiting for the ensemble. A node with more memory, such as an 8 GB module, could keep all engines resident and answer synchronously; the server implements this mode, but we have not tested it.

Portability is more than the model. Two of the deviations we measured did not come from the models at all. The edge server initially resized images bicubically while the edge model had been trained with bilinear resizing, and the Nano's older image library decodes JPEGs differently (Section 5.5). Both shifted predictions without any error being raised. Verifying deployed nodes against offline predictions image by image, as in Table 5, is what exposed them; we recommend the practice for any edge–cloud deployment.

Limitations. Our study uses a single, if diverse, benchmark; PlantWild's class balance and capture conditions may differ from a given region. The escalation threshold is global; per-crop thresholds could equalise accuracy across farms. The emulation compresses idle time and therefore does not capture diurnal effects, power cycling or connectivity outages, and its four farms share one physical Pi, so device-to-device variability is not represented. Each edge model was trained once, so we report no variance across training seeds for the edge models and the distillation; the bootstrap of Section 6.2 covers the variability of the calibrated threshold, and the loop pilot is repeated over 5 random partitions. The escalation threshold is a point estimate whose test result falls on either side of the target, and the benchmark has no second field dataset with the same classes on which to test it, so we do not claim a guarantee beyond PlantWild. In the prototype the farmer's answer to an escalated image waits for the upload to be acknowledged (Section 4.3). The Pi's energy covers its SoC and memory rails only and is therefore a lower

bound. In our testbed the devices share a local network, so measured network latencies are a best case for rural links; transfer times on slower links follow directly from the measured byte counts. The confirmation delay of the batched cloud tier is a design parameter set in simulated time (two batches a day), not a measured quantity; its compute and loading costs are measured. Finally, the Nano's cloud pipeline deviates slightly from the workstation's, chiefly through its older image library; Table 5 and the deployment-wide agreement quantify the deviation.

8. Conclusion

Selective escalation reconciles the accuracy of foundation-model ensembles with the connectivity of real farms. A MobileNetV3-Large edge model that forwards an image only when its two leading hypotheses are close — with a threshold fixed on validation data — retains 94.6 % of a three-backbone ensemble's accuracy on held-out in-the-wild plant disease images while uploading 5.4 times less data. Edge-model capacity, not router complexity, sets this trade-off, and a cloud-supervised verifier pays for its own message without improving routing. The complete system runs on a Raspberry Pi 5 and an end-of-life 4 GB Jetson Nano, made possible by a port that decouples the ensemble from its framework and runs it one chunk engine at a time, at 49 J per escalated image; a seven-day, four-farm emulated deployment on that hardware confirms the offline trade-off: 18.6 % of images escalated, uploaded bytes cut 5.4-fold, and 76.0 % of diagnoses correct, with a median answer time on the farm of 77 ms and 5.1 times less device energy per diagnosis than always-cloud. Distilling the ensemble into the edge model, once and at no inference cost, raises the uplink saving measured on the same devices from 5.4- to 8.0-fold (7.5 KB per diagnosis) at 75.3 % delivered accuracy — and to 7.0-fold at unchanged accuracy — while cutting device energy per diagnosis from 9.5 to 6.4 J. Future work will accumulate escalations over full seasons to test whether the loop from cloud to edge improves farms whose images differ from the training data, a gain one week of in-distribution escalations could not show; it will also tune thresholds per crop and study connectivity outages over full-length deployments.

CRediT authorship contribution statement

Youssef Natij, Hajar El Karch, Abdelkader Mezouari: Conceptualization, Methodology, Software, Validation, Investigation, Formal analysis,

Writing – original draft, Writing – review & editing. All authors contributed equally to the conception, implementation, analysis and writing of this work.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Funding

This research did not receive any specific grant from funding agencies in the public, commercial, or not-for-profit sectors.

Data availability

PlantWild is available from its authors [10] under the terms of their release; this work does not redistribute its images. Everything needed to reproduce the tables and figures of this paper is in the project repository, `https://github.com/YouROS12/CloudOffEdge`: the deployment code for the Raspberry Pi and the Jetson Nano, the porting toolkit (chunked ONNX export, fp16 conversion, chunk-by-chunk TensorRT build, chunk-major runtime), the fleet emulator and operations console, the training, distillation, calibration and measurement scripts, the trained edge models and cloud head, the per-image predictions, the per-image logs of both emulated deployments, the activation, memory and power profiles, the TensorRT verification reports and the sources of this manuscript.

Declaration of generative AI and AI-assisted technologies in the manuscript preparation process

During the preparation of this work the authors used Claude (Anthropic) to assist with software development and with drafting and editing the manuscript. After using this tool, the authors reviewed and edited the content as needed and take full responsibility for the content of the publication.

Appendix A. Software adaptations for embedded deployment

The reference pipeline was developed on a workstation (Python 3.12, PyTorch 2.11, `timm` 1.0.26, `open_clip` 3.3.0). Running it on the embedded testbed required the adaptations below, listed so that the deployment can be reproduced on the same hardware generation. Each adaptation was verified against the reference pipeline (Table 5).

Table A.1: Software adaptations, by node.

#	Problem and cause	Adaptation
Edge node — Raspberry Pi 5, Debian 13, Python 3.13		
A1	The edge requirements originally pinned PyTorch 2.2 and NumPy 1.26, which have no Python 3.13 wheels for ARM64 (the workstation reference itself uses PyTorch 2.11).	Version-conditional pins (PEP 508 markers): original pins below 3.13, PyTorch 2.6 / NumPy 2.2 from 3.13.
A2	One physical device must emulate several farms.	systemd template unit; one independent server process per farm with its own identifier and port.
A3	Latency reported only as a total.	Edge reports its own inference time; the cloud reports per-backbone times; network time is the remainder.
A4	A cloud outage would appear as a low escalation rate.	Escalation decisions logged separately from cloud success; failed escalations fall back to the edge label.
A5	The edge server resized images bicubically, whereas the model was trained and evaluated with bilinear resizing, which shifts the margins.	Bilinear resizing restored; the deployed node reproduces the offline prediction on 300/300 test images.
A6	Escalated images were uploaded at camera resolution (up to $\sim$6 MB), although the cloud works at $\leq$336 px.	Payload resized to a 448 px shorter side, JPEG q90 (Table 2); the always-cloud baseline uses the same payload.
A7	A failed escalation was billed as uploaded bytes.	Bytes are counted only once the cloud acknowledges the image.

#	Problem and cause	Adaptation
Cloud node — Jetson Nano 4 GB, JetPack 4.6.1, Python 3.6, CUDA 10.2, TensorRT 8.2.1		
B1	The reference server needs Python ≥3.8 (PyTorch 2, `timm` for DINOv3), unavailable on JetPack 4.	Model decoupled from framework: ONNX export on the workstation, TensorRT engines built on the Nano, torch-free Python 3.6 server.
B2	TensorRT 8.2 parses ONNX opsets up to 13.	Export at opset 13 with the TorchScript-based exporter.
B3	Fused scaled-dot-product attention has no opset-13 mapping.	Explicit softmax$(QK^\top/\sqrt{d})V$ substituted during export (mathematically identical).
B4	The `nn.MultiheadAttention` fast path (CLIP) emits a fused operator with no ONNX mapping.	Fast path disabled during export.
B5	DINOv2 bicubically resizes its position grid on every forward; TensorRT 8.2 has no cubic resize.	Input size is fixed, so the interpolated position embedding is computed once in PyTorch and stored as a constant.
B6	DINOv3 recomputes its rotary position embedding from the input shape on every forward, and CLIP's reshapes emit dynamic shape arithmetic; both export as conditionals (`If`) and modulo operators.	Rotary embedding baked as a constant for the fixed 21×21 patch grid; remaining shape arithmetic constant-folded (onnx-simplifier); export rejected if an unsupported operator remains.
B7	No PyTorch/torchvision in the serving process.	Preprocessing re-implemented with Pillow (JPEG decoding, bicubic resize, as in torchvision) and NumPy (crop, normalisation, layout), bit-identical on the workstation; QualityGate head in NumPy (logits within 3×10^{-6}).
B8	Engine builds were killed for lack of memory: 4 GB is shared by CPU and GPU, and the stock swap is compressed RAM, in which weights barely compress.	One engine built per process; the graph parsed directly from file (one copy of the weights); fp16 with a 256 MB builder workspace; a temporary swap file on the SD card for the build only.

#	Problem and cause	Adaptation
B9	Even so, building from fp32 graphs exhausted the memory the GPU can allocate (which cannot be swapped).	Graphs converted to fp16 weights on the workstation (halving them to ∼607 MB), with LayerNorm arithmetic kept in fp32 because ViT activation outliers overflow fp16 when squared; cosine to the fp32 reference ≥ 0.9999 in onnxruntime, whose CPU provider computes these graphs in fp32 — the check covers the rounding of the weights, not fp16 arithmetic (see B14).
B10	A whole fp16 ViT-L still could not be built: parsing alone peaks at 2.75 GB, and the build thrashed swap with the GPU idle.	Each backbone exported as four chunks of six transformer blocks, built one engine at a time and chained at run time (exact: cosine 1.000000 to the unsplit model).
B11	TensorRT 8.2 compiled each chunk into one fused kernel graph holding 969 MB of device memory (13 bytes per parameter).	Every linear layer exported as an equivalent 1×1 convolution, applied to the (batch, tokens, channels) residual stream reshaped to $1 \times C \times (B\,N) \times 1$ and reshaped back (attention for CLIP rewritten explicitly); engine size $612 \rightarrow 157$ MB, no fused graphs remain.
B12	Chunk builds took ∼10 min each.	A timing cache shared across chunks, whose layer shapes are identical within a backbone.
B13	Only nine of the twelve chunk engines fit in memory (measured with fp16 engines), even with one shared activation buffer; fp32 engines are twice as large.	Chunk-major batched cloud inference with deferred escalation (Section 5.4); one chunk engine resident at a time.

#	Problem and cause	Adaptation
B14	First parity check failed (top-1 agreement 1/24): DINOv3 features were NaN. Pinning LayerNorm, then all non-convolution layers, to fp32 changed nothing; profiling showed DINOv3's residual stream reaching 1.6×10^5, beyond the fp16 range.	Per-backbone precision: DINOv2 fp16, DINOv3 fp32; CLIP also fp32, chosen while its deviation (B15) was still attributed to precision (a later check showed fp16 CLIP equivalent and faster). Every layer pinned with TensorRT precision constraints, since the fp16-weight graph requires the fp16 builder flag: the fp32 engines keep the fp16-rounded weights but compute in fp32. The workstation check of B9 had not shown the overflow because onnxruntime computed in fp32 (the first DINOv3 chunk's hidden state reaches 1.57×10^5 there without overflowing).
B15	CLIP still deviated (cosine below 0.99 on 5/24 reference images), identically in fp16 and fp32.	Traced to the image library: Pillow 8.4 (the last release for Python 3.6) links IJG libjpeg 9, Pillow 12 libjpeg-turbo 3.1. The decoders differ on 36 % of pixels (up to 67/255); resampling the same decoded pixels is bit-identical in both versions. Given identical input tensors the CLIP and DINOv3 engines match the reference (cosine 0.99999–1.00000) and all engines agree on 24/24 top-1. Accepted rather than corrected (a matched decoder is not available as a stock JetPack 4 package); measured over the emulated deployments.
B16	TensorRT Python bindings ship with JetPack, not PyPI.	Virtual environment with access to system site-packages.
B17	Current Python packages no longer support Python 3.6.	Last compatible releases pinned (NumPy 1.19.5, Pillow 8.4.0, pycuda 2021.1, FastAPI 0.83, pydantic 1.9).

#	Problem and cause	Adaptation
B18	`import numpy` aborts with an illegal instruction: the bundled OpenBLAS mis-detects the Cortex-A57.	`OPENBLAS_CORETYPE=ARMV8` set for every process.
B19	pycuda's isolated build compiles NumPy from source (∼20 min on the Nano).	NumPy installed from a wheel first, pycuda built without build isolation.
B20	The shared message definitions require Python ≥3.7.	Message models mirrored in pydantic v1; the wire format is unchanged, so edge nodes need no change.
B21	The web server runs requests on a thread pool; CUDA contexts are thread-bound.	One CUDA context shared by all engines, pushed and popped around each inference under a lock.
Workstation and orchestration		
C1	Scripts edited on Windows carried CRLF line endings, which break Bash on the devices.	LF enforced through `.gitattributes` and normalised before shipping.
C2	Devices hold no credentials for the private code repository.	Code shipped over SSH from the workstation; checkpoint integrity checked by SHA-256.
C3	The paper's test split had to be reproduced exactly.	The seed-fixed 3 677-image split materialised as hard links in a class-per-folder layout.
C4	The distilled configuration (Section 6.7) had to run on the same devices without losing the baseline one.	Distilled edge checkpoint and its own validation-calibrated threshold selected through a separate configuration file; fp16 CLIP engines in a separate engine directory selected by a service override; both reversible. The deployed edge reproduces the distilled model's offline predictions on 300/300 test images.

#	Problem and cause	Adaptation
C5	Added after the measurements: a queue that accumulates for long (more farms, a missed batch) would hold every image's hidden state at once, about 4.5 MB per image.	The cloud server processes queues of more than 128 images in sub-batches of 128; the edge server can optionally upload escalations from a background queue so that the answer never waits for the network. Neither affects the reported runs (largest batch 61 images; uploads acknowledged before answering).

References

[1] S. Savary, L. Willocquet, S. J. Pethybridge, P. Esker, N. McRoberts, A. Nelson, The global burden of pathogens and pests on major food crops, Nature Ecology & Evolution 3 (3) (2019) 430–439. doi:10.1038/s41559-018-0793-y.

[2] A. Kamilaris, F. X. Prenafeta-Boldú, Deep learning in agriculture: A survey, Computers and Electronics in Agriculture 147 (2018) 70–90. doi:10.1016/j.compag.2018.02.016.

[3] S. P. Mohanty, D. P. Hughes, M. Salathé, Using deep learning for image-based plant disease detection, Frontiers in Plant Science 7 (2016) 1419. doi:10.3389/fpls.2016.01419.

[4] K. P. Ferentinos, Deep learning models for plant disease detection and diagnosis, Computers and Electronics in Agriculture 145 (2018) 311–318. doi:10.1016/j.compag.2018.01.009.

[5] D. P. Hughes, M. Salathé, An open access repository of images on plant health to enable the development of mobile disease diagnostics, arXiv preprint arXiv:1511.08060 (2015). doi:10.48550/arXiv.1511.08060.

[6] J. G. A. Barbedo, Impact of dataset size and variety on the effectiveness of deep learning and transfer learning for plant disease classification, Computers and Electronics in Agriculture 153 (2018) 46–53. doi:10.1016/j.compag.2018.08.013.

[7] M. A. Noyan, Uncovering bias in the PlantVillage dataset, arXiv preprint arXiv:2206.04374 (2022). doi:10.48550/arXiv.2206.04374.

[8] A. Ahmad, A. E. Gamal, D. Saraswat, Toward generalization of deep learning-based plant disease identification under controlled and field conditions, IEEE Access 11 (2023) 9042–9057. doi:10.1109/ACCESS.2023.3240100.

[9] X. Wu, X. Fan, P. Luo, S. D. Choudhury, T. Tjahjadi, C. Hu, From laboratory to field: Unsupervised domain adaptation for plant disease recognition in the wild, Plant Phenomics 5 (2023) 0038. doi:10.34133/plantphenomics.0038.

[10] T. Wei, Z. Chen, Z. Huang, X. Yu, Plant disease recognition in the wild, in: Proceedings of the 32nd ACM International Conference on Multimedia (MM '24), 2024. doi:10.1145/3664647.3681614.

[11] M. Oquab, T. Darcet, T. Moutakanni, H. V. Vo, M. Szafraniec, V. Khalidov, P. Fernandez, D. Haziza, F. Massa, A. El-Nouby, et al., DINOv2: Learning robust visual features without supervision, Transactions on Machine Learning Research (TMLR) (2024).

[12] O. Siméoni, H. V. Vo, M. Seitzer, F. Baldassarre, M. Oquab, et al., DINOv3: A versatile vision foundation model, Transactions on Machine Learning Research (TMLR) (2026).
URL https://openreview.net/forum?id=2NlGyqNjns

[13] A. Radford, J. W. Kim, C. Hallacy, A. Ramesh, G. Goh, S. Agarwal, G. Sastry, A. Askell, P. Mishkin, J. Clark, G. Krueger, I. Sutskever, Learning transferable visual models from natural language supervision, in: International Conference on Machine Learning (ICML), 2021, pp. 8748–8763.

[14] H. El Karch, Y. Natij, M. Benaly, R. El Gouri, A. Mezouari, Backbone diversity beats text supervision: A systematic study of frozen multi-foundation model fusion for in-the-wild plant disease recognition, Frontiers in Plant Science 17 (2026) 1860665. doi:10.3389/fpls.2026.1860665.

[15] A. Howard, M. Sandler, G. Chu, L.-C. Chen, B. Chen, M. Tan, W. Wang, Y. Zhu, R. Pang, V. Vasudevan, Q. V. Le, H. Adam, Searching for

MobileNetV3, in: IEEE/CVF International Conference on Computer Vision (ICCV), 2019, pp. 1314–1324. doi:10.1109/ICCV.2019.00140.

[16] S. Wolfert, L. Ge, C. Verdouw, M.-J. Bogaardt, Big data in smart farming – a review, Agricultural Systems 153 (2017) 69–80. doi:10.1016/j.agsy.2017.01.023.

[17] M. J. O'Grady, D. Langton, G. M. P. O'Hare, Edge computing: A tractable model for smart agriculture?, Artificial Intelligence in Agriculture 3 (2019) 42–51. doi:10.1016/j.aiia.2019.12.001.

[18] W. Shi, J. Cao, Q. Zhang, Y. Li, L. Xu, Edge computing: Vision and challenges, IEEE Internet of Things Journal 3 (5) (2016) 637–646. doi:10.1109/JIOT.2016.2579198.

[19] P. Viola, M. Jones, Rapid object detection using a boosted cascade of simple features, in: Proceedings of the IEEE Computer Society Conference on Computer Vision and Pattern Recognition (CVPR), Vol. 1, 2001, pp. I–511–I–518. doi:10.1109/CVPR.2001.990517.

[20] C. K. Chow, On optimum recognition error and reject tradeoff, IEEE Transactions on Information Theory 16 (1) (1970) 41–46. doi:10.1109/TIT.1970.1054406.

[21] S. Teerapittayanon, B. McDanel, H. T. Kung, BranchyNet: Fast inference via early exiting from deep neural networks, in: 23rd International Conference on Pattern Recognition (ICPR), 2016, pp. 2464–2469. doi:10.1109/ICPR.2016.7900006.

[22] D. Singh, N. Jain, P. Jain, P. Kayal, S. Kumawat, N. Batra, PlantDoc: A dataset for visual plant disease detection, in: Proceedings of the 7th ACM IKDD CoDS and 25th COMAD, 2020, pp. 249–253. doi:10.1145/3371158.3371196.

[23] T. Wei, Z. Chen, Z. Huang, X. Yu, Benchmarking in-the-wild multimodal disease recognition and a versatile baseline, in: Proceedings of the 32nd ACM International Conference on Multimedia (MM '24), 2024, pp. 1593–1601. doi:10.1145/3664647.3680599.

[24] X. Jiang, J. Wang, K. Xie, C. Cui, A. Du, X. Shi, W. Yang, R. Zhai, PlantCaFo: An efficient few-shot plant disease recognition method based

on foundation models, Plant Phenomics 7 (2025) 100024. doi:10.1016/j.plaphe.2025.100024.

[25] Z. Salman, A. Muhammad, D. Han, Plant disease classification in the wild using vision transformers and mixture of experts, Frontiers in Plant Science 16 (2025). doi:10.3389/fpls.2025.1522985.

[26] M. J. Hasan, S. Mazumdar, S. Momen, Deployable deep learning for cross-domain plant leaf disease detection via ensemble learning, knowledge distillation, and quantization, IEEE Access 13 (2025) 140313–140336. doi:10.1109/ACCESS.2025.3595390.

[27] Y. Kang, J. Hauswald, C. Gao, A. Rovinski, T. Mudge, J. Mars, L. Tang, Neurosurgeon: Collaborative intelligence between the cloud and mobile edge, in: Proceedings of the 22nd International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS), 2017, pp. 615–629. doi:10.1145/3037697.3037698.

[28] S. Teerapittayanon, B. McDanel, H. T. Kung, Distributed deep neural networks over the cloud, the edge and end devices, in: IEEE 37th International Conference on Distributed Computing Systems (ICDCS), 2017, pp. 328–339. doi:10.1109/ICDCS.2017.226.

[29] Y. Matsubara, M. Levorato, F. Restuccia, Split computing and early exiting for deep learning applications: Survey and research challenges, ACM Computing Surveys 55 (5) (2022) 1–30. doi:10.1145/3527155.

[30] Y. Geifman, R. El-Yaniv, Selective classification for deep neural networks, in: Advances in Neural Information Processing Systems (NeurIPS), 2017.

[31] D. Hendrycks, K. Gimpel, A baseline for detecting misclassified and out-of-distribution examples in neural networks, in: International Conference on Learning Representations (ICLR), 2017.

[32] C. Guo, G. Pleiss, Y. Sun, K. Q. Weinberger, On calibration of modern neural networks, in: Proceedings of the 34th International Conference on Machine Learning (ICML), Vol. 70 of Proceedings of Machine Learning Research, 2017, pp. 1321–1330.

[33] X. Wang, Y. Luo, D. Crankshaw, A. Tumanov, F. Yu, J. E. Gonzalez, IDK cascades: Fast deep learning by learning not to overthink, in: Proceedings of the Conference on Uncertainty in Artificial Intelligence (UAI), 2018.

[34] A. Kag, I. Fedorov, A. Gangrade, P. Whatmough, V. Saligrama, Efficient edge inference by selective query, in: International Conference on Learning Representations (ICLR), 2023.

[35] M. Tan, Q. V. Le, EfficientNet: Rethinking model scaling for convolutional neural networks, in: International Conference on Machine Learning (ICML), 2019, pp. 6105–6114.

[36] I. Loshchilov, F. Hutter, Decoupled weight decay regularization, in: International Conference on Learning Representations (ICLR), 2019.

[37] T.-Y. Lin, P. Goyal, R. Girshick, K. He, P. Dollár, Focal loss for dense object detection, in: IEEE International Conference on Computer Vision (ICCV), 2017, pp. 2999–3007. doi:10.1109/ICCV.2017.324.

[38] J. A. Donenfeld, WireGuard: Next generation kernel network tunnel, in: Network and Distributed System Security Symposium (NDSS), 2017. doi:10.14722/ndss.2017.23160.

[39] J. Bai, F. Lu, K. Zhang, et al., ONNX: Open neural network exchange, https://github.com/onnx/onnx, accessed 2026-10-11 (2019).

[40] NVIDIA Corporation, NVIDIA TensorRT 8.2 developer guide, https://docs.nvidia.com/deeplearning/tensorrt/archives/, accessed 2026-10-11 (2022).

[41] G. Hinton, O. Vinyals, J. Dean, Distilling the knowledge in a neural network, arXiv preprint arXiv:1503.02531 (2015). doi:10.48550/arXiv.1503.02531.